

# 暨南大学硕士学位论文

题名（中英对照）：

**基于 NAND 闪存物理特性的读干扰管理与可靠性研究**

**Research on read disturb management and reliability based on physical  
characteristics of NAND Flash Memory**

作者姓名：**张 根 雄**

指导教师姓名及学位、职称：**邓玉辉 博士 教授**

学科、专业名称：**计算机技术**

学位类型：**专业学位**

论文提交日期：**2023 年 6 月**

论文答辩日期：**2023 年 5 月**

答辩委员会主席：**李仁发**

论文评阅人：**盲审**

学位授予单位和日期：**暨南大学 6 月 21 日**



# 独 创 性 声 明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 暨南大学 或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

学位论文作者签名：张根雄

签字日期：2023 年 6 月 19 日

# 学位论文版权使用授权书

本学位论文作者完全了解 暨南大学 有关保留、使用学位论文的规定，有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权 暨南大学 可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

(保密的学位论文在解密后适用本授权书)

学位论文作者签名：张根雄

签字日期：2023 年 6 月 19 日

学位论文作者毕业后去向：

工作单位：

通讯地址：

导师签名：张根雄

签字日期：2023 年 6 月 19 日

电 话：

邮 编：



## 摘 要

全球信息数字化、智能终端、物联网规模的迅速扩张，导致了数据存储总量的爆炸型增长。3D NAND 闪存固态硬盘凭借低读写时延、大容量、高并行、低能耗等优点逐渐成为主要的存储介质。然而，随着浮栅晶体管和电荷陷阱晶体管等介质工艺技术和 3D 堆叠技术对更高存储密度的追求，研究如何保证闪存的可靠性已经成为当前存储系统中最重要的挑战之一。近年来，大量读密集型场景的出现致使读干扰成为了影响闪存系统可靠性不可忽视的主要问题。造成读干扰的主要原因是读累积。本文围绕闪存系统的读干扰缓解策略展开研究，提出可以结合闪存写入数据的冷热属性和闪存的物理特性对数据进行管理。管理方案从不同结构层级上区别看待读干扰，包含冷热数据混合放置策略和热数据缓存策略。

针对热读场景下的严重读重写问题，本文首先提出一种名为 Cocktail 的数据放置策略。严重读干扰会使得闪存块中数据的读取时间大大增加甚至造成数据丢失，因此闪存系统中存在一种名为读重写的机制，该机制类似于垃圾回收操作，会将闪存块中的有效数据写到其它位置并对原闪存块进行擦除操作，通过这种方式将数据从严重读干扰中解放出来。Cocktail 指出热读数据的聚集是造成读重写的主要原因，由此提出可以设计数据过滤策略，在用户写入的数据中提取出部分冷数据，同时维护一个数据写入池和控制冷热数据的写入比例，让多个闪存块一起分担热读数据产生的读干扰。本文将 Cocktail 与现有的三种方案 double write frontier、redFTL 和 IPR 进行对比，实验结果表明，与现有方案相比，Cocktail 的读重写次数、平均响应时间、99% 尾延迟、GC 次数和写放大平均减少了 40.77%、10.82%、5.40%、12.29% 和 49.57%。

针对部分数据的频繁读取问题，本文提出了一种读干扰感知型的数据缓存策略 Minato。闪存转换层会在有限的 DRAM 空间中维护一个读/写缓冲区，频繁访问的数据可以存储在其中减少反应时间。由于垃圾回收和更长的执行时间，频繁写请求对 NAND 闪存的影响往往比频繁读请求更大。因此，在为读缓存分配 DRAM 空间时必须保证尽量不影响写缓存的命中率。Minato 设计了一种 DRAM 分配策略，可以在不影响 DRAM 接收写请求的前提下为读缓存分配空间。之后，Minato 通过监测闪存块的读干扰情况，为读缓存设置了一个数据过滤器，当热数据较多且空间不够时，优先缓存读干扰影响较大的数据，从而降低闪存整体的读干扰水平。实验部分在九个实际工作负载下测量了 Minato 的写请求缓存命中率和整体读干扰水平，实验结果表明，Minato 可以在不影响写请求命中率的情况下缓解

读干扰、提升读性能。与单纯的读缓存策略相比，Minato 的整体读干扰水平、读重写次数、读反应时间最高可减少 3.68%、15.32%、13.14%。

**关键词：** NAND Flash；读干扰；固态硬盘；缓存策略

## ABSTRACT

As global information digitization, smart terminals, and the Internet of Things proliferate at a rapid pace, the total amount of data storage has skyrocketed. Owing to its compact size, low-power consumption, shock resistance, and fast access speed, 3D NAND flash memory has emerged as the predominant storage medium in recent years. Ensuring the reliability of flash memory has become a paramount challenge in current storage systems, as process technologies such as floating gate transistors and charge trap transistors and 3D stacking technology strive for higher storage density. Read disturbance has emerged as a major problem that impairs the reliability of flash memory systems in recent years, due to the proliferation of read-intensive scenarios. The root cause of read disturbance is read accumulation. This paper proposes a novel data management scheme that leverages the cold and hot attributes of flash memory write data and the physical characteristics of flash memory to mitigate read disturbance for flash memory systems. The scheme differentiates read disturbance at different structural levels and adopts cold and hot data mixed placement strategy and buffer management.

First, this thesis proposes a novel read-disturb management approach called Cocktail that mixes a small proportion of hot-read pages with a large proportion of cold-read pages, thereby avoiding clustering hot-read pages into a few blocks. Motivated by the insight that read reclaim operations are frequently triggered by hot read-pages, Cocktail first prefills a portion of each block with cold data extracted from user requests. Cocktail fills the prefilled blocks with write-back data caused by read reclaim to create read-balanced blocks. We integrate two thresholds, write pool capacity and the ratio of RR-write data to User-write data, into Cocktail to govern the ratio of write-back data caused by read reclaim to data of user requests in a block. Cocktail dynamically adjusts the two thresholds according to the characteristics of read reclaim. Cocktail is conducive to decentralizing hot write-back data caused by read reclaim across a broad range of blocks, thereby reducing the occurrence of second-time read reclaim and the number of overall block reads. We compare Cocktail with three existing schemes baseline, redFTL, and IPR in terms of SSD service life, SSD response time, write amplification, and the number of GCs under ten real-world workload conditions. Experimental results show that compared with the existing schemes, Cocktail reduces the number of read reclaims, the average response time, the 99-percentile tail latency, and the

number of GCs by an average of 40.77%, 10.82%, 5.40%, and 12.29%, respectively. Cocktail also alleviates the write amplification of the three alternative schemes by an average of 49.57%.

Second, this thesis proposes a read disturb-aware buffer management scheme called Minato, which absorbs the read requests based on the read-disturb level of blocks to reduce the overall read disturbance. In order to reduce response time, flash system stores frequently accessed data in a read/write buffer within a limited DRAM space. Writes have a greater impact on NAND flash memory than reads due to “garbage collection” and “unbalanced execution time”, flash system needs to ensure that the hit rate of write buffer is not affected as much as possible while allocating DRAM space for read buffer. Minato contains a DRAM allocation strategy that can allocate space for read buffer without affecting DRAM receiving write requests. Minato contains a data filter for read buffer by monitoring the read disturbance situation of flash blocks. To reduce the overall read-disturb level of flash memory, it prioritizes caching data that have a higher read disturbance impact when there are more hot data than available space. The experiment evaluates write request cache hit rate and overall read disturbance level under nine real-world workloads. The results show that Minato alleviates the overall read disturbance of flash memory without affecting the writes hit ratio. Compared with the read-buffer strategy, Minato reduces the overall read-disturb level, read reclaim times, and read response time by 3.68%, 15.32%, and 13.14% in the best case.

**Keywords:** NAND Flash; read disturb; solid state disk; buffer management



# 目 录

|                          |       |
|--------------------------|-------|
| 摘 要                      | (I)   |
| ABSTRACT                 | (III) |
| 目 录                      | (V)   |
| 1 绪论                     | (1)   |
| 1.1 研究背景及意义              | (1)   |
| 1.2 国内外研究现状              | (3)   |
| 1.3 研究主要内容               | (6)   |
| 1.4 论文章节安排               | (7)   |
| 2 NAND 闪存存储器概述           | (8)   |
| 2.1 NAND 闪存结构及工作原理       | (8)   |
| 2.2 闪存控制器                | (12)  |
| 2.3 NAND 闪存可靠性分析         | (14)  |
| 2.4 本章小结                 | (16)  |
| 3 基于访问行为的数据放置策略 Cocktail | (17)  |
| 3.1 引言                   | (17)  |
| 3.2 NAND 闪存数据访问行为分析      | (19)  |
| 3.3 Cocktail 的设计与实现      | (20)  |
| 3.4 实验验证及分析              | (32)  |
| 3.5 本章小结                 | (39)  |
| 4 基于读干扰感知的数据缓存策略 Minato  | (40)  |
| 4.1 引言                   | (40)  |
| 4.2 NAND 闪存读干扰影响情况分析     | (42)  |
| 4.3 Minato 的设计与实现        | (44)  |
| 4.4 实验验证及分析              | (51)  |
| 4.5 本章小结                 | (58)  |

5 总结与展望 ..... (59)

5.1 本文工作总结 ..... (59)

5.2 未来工作展望 ..... (60)

参考文献 ..... (61)

在学期间发表的学术论文及科研成果清单 ..... (69)

致 谢 ..... (70)

# 1 绪论

## 1.1 研究背景及意义

由于信息数字化、智能终端、物联网规模的迅速扩张,导致了全球数据存储总量呈爆炸式增长。根据国际著名咨询机构 IDC 的报告,到 2025 年,全球数据总量将达到 175ZB,中国将成为数据总量排名世界第一的国家。而且,全球数据总量的年增长率将维持在 50% 左右。在这其中,26% 的数据将会被存储在闪存固态硬盘中。相比于机械硬盘,闪存固态硬盘有着低读写时延、高并行、低能耗等优点。而且,得益于浮栅晶体管和电荷陷阱晶体管等介质工艺技术<sup>[1]</sup>和 3D 堆叠技术<sup>[2]</sup>的发展,固态硬盘一直在追求更高的存储密度和更低的每比特成本<sup>[3]</sup>。然而,可靠性问题和寿命问题已经成为限制固态硬盘发展最主要的问题之一。固态硬盘中的数据写入基于异地更新操作,即旧数据会成为失效数据,要覆写失效数据所在空间时需要先对闪存块执行擦除操作。这样的擦除操作会磨损闪存单元<sup>[4-5]</sup>,当闪存单元被磨损到无法正确读取数据时将不能继续进行存储服务,严重时甚至会导致服务器关机和丢失数据。Jaffer 等人<sup>[6]</sup>指出,随着存储密度的提高,固态硬盘的寿命越来越低。以 5nm 制程工艺为例,SLC 闪存和 MLC 闪存的寿命约为 10000 个擦除周期,而 TLC 闪存的寿命约为 2000 个擦除周期,QLC 闪存的寿命甚至不足 500 个擦除周期。而且, Park 等人<sup>[7]</sup>指出,随着擦除次数的增加,单次读取的数据错误率也随之增加。例如在 30 摄氏度的温度条件下,同一闪存固态硬盘在 2000 次擦除时的读取错误比特数约为 0 次擦除时的 2 倍。为了持续进行存储服务以及避免发生数据丢失,各大数据中心都需要付出大量的维护成本来应对硬盘损耗。因此,在闪存固态硬盘应用如此广泛的大环境下,研究如何提高闪存寿命和闪存可靠性具有战略上的意义。

闪存固态硬盘在其控制器中存在一个 ECC 校验机制,存储数据的同时也存储数据的 ECC 校验信息。ECC 校验机制可以在读取数据时检测和纠正大量的原始比特错误,一旦原始比特误码率超过了 ECC 可以纠正的最大错误数可能会发生数据丢失。提高闪存寿命和闪存可靠性的主要优化目标就是减少原始比特错误,延长数据错误率超出 ECC 校验能力的时间。造成闪存固态硬盘数据读取错误的原因主要包括闪存单元磨损、编程干扰、读干扰、保留时间过长造成的电荷泄漏等<sup>[7]</sup>。由于大量读密集型场景的出现,读干扰成为了影响闪存固态硬盘可靠性和寿命不可忽视的主要问题。读干扰问题的主要原因是读累积。现代 NAND 闪存物理介质的组织结构自上而下分别为通道 (channel)、芯片 (chip)、颗粒

(die)、闪存片 (plane)、闪存块 (block)、闪存页 (page)。其中, 一个闪存块中包含多个字线 (WL), 每个字线上又包含多个闪存页<sup>[8]</sup>。当读取一个闪存块中的 WL 时, 闪存必须对同一闪存块中的所有其它 WL 施加一个直通电压, 相当于一个弱编程操作。这样的读取操作会严重干扰闪存块的电压分布, 从而产生更多的比特错误。而且, 以 TLC 闪存为例, 闪存块中一个字线上有三个页面 P0, P1 和 P2, 分别称为低页 (LSB)、中页 (CSB)、高页 (MSB)。读取低页只需施加一次电压, 读取中页需要施加两次电压, 读取高页需要施加三次电压。读取高页不仅需要施加更多次的读电压, 产生比特错误的可能性也更高。当 ECC 无法正确校验数据时, 闪存必须改变电压并且重复执行同一读取操作, 直到被干扰页面的数据被正确读取。因此, 当读干扰严重时, 闪存读取一个页面需要多花几倍的时间。更糟糕的情况是, 当多次读取后依然没有获得正确的数据时, 闪存将永久丢失该页面中的数据, 严重影响闪存的可靠性。

为了避免数据丢失, 闪存中存在一种称为读重写的机制<sup>[7]</sup>。读重写操作类似于垃圾回收机制<sup>[9-10]</sup>, 即当一个闪存块存在严重读干扰时, 闪存会将该闪存块中的有效数据重写到其它位置, 并对该闪存块执行擦除操作。重写到其它位置的数据将从被严重干扰的状态中恢复过来。在读密集型场景下, 闪存可能需要触发大量的读重写 (擦除) 操作来消除读干扰。Liu 等人<sup>[11]</sup>在 8 个最先进的闪存上执行了多达 200 天的各种随机读工作负载, 实验表明闪存存在后台执行了大量的擦除操作来修复被干扰的数据。并且, 在 56 MBps 随机读吞吐量条件下, 一个 3D QLC 闪存在大约 427 天内损坏, 这比它的 3 年保修期短得多。

一般来说, 闪存块因为严重读干扰而导致读延迟增加甚至发生读重写的原因主要是热读数据在闪存块上的聚集。数据按其一定时间范围内读取访问的频度可以分为热读数据、温读数据、冷读数据。当闪存块中大部分页面都是热读数据时, 需要在各条字线上反复施加直通电压, 从而影响闪存单元的电压分布, 造成数据读取错误。如果热读数据放置不合理, 必然会在闪存中形成较多的热读闪存块。这样的热读闪存块会产生严重的读干扰, 延长读反应时间, 闪存控制器不得不执行额外的擦除操作来保证可靠性和数据完整性, 严重缩短闪存寿命。

如果能在闪存所有写入数据中区分出热读数据和冷读数据, 控制它们在相同闪存块中的放置比例, 使得闪存块在被正常垃圾回收之前不会因为读干扰而执行额外的读重写操作, 则将减少许多额外的擦除操作。例如: 假设一个闪存块有 10 个页面, 每个页面读取 10 次即执行读重写操作。假设闪存块中都是有效的热读数据。如果将 10 个热读页面都回写到相同的闪存块中, 则在每个页面重新读取 10 次之后该闪存块将再次发生读重写。然而, 如

果将 10 个热读页面回写到 10 个不同的闪存块中, 并且和其它冷读数据结合, 则当一个闪存块中有一个热读页面读取 10 次时, 该闪存块总的读电压施加次数可能还远远没达到触发读重写操作的阈值。之后, 该闪存块如果正常被垃圾回收, 则其中的数据也将从读干扰中恢复过来, 无需进行额外的读重写操作。另外, 闪存系统中存在一个 DRAM 缓存空间, 用来存放逻辑地址和物理地址的映射表以及作为读写缓存。直观上来说, 在 DRAM 中设立一个读缓存, 将热读数据缓存到读缓存中是缓解读干扰的一个有效手段。

因此, 在读密集型场景中, 如果能综合考虑数据的访问行为和闪存的物理特性对读干扰的影响来在闪存中进行数据放置和数据缓存, 则能有效缩短反应时间、减少读干扰, 从而减少闪存中额外的擦除操作, 延长闪存的使用寿命。基于以上分析, 本文将研究探索读密集场景下基于访问行为的数据放置策略和基于读干扰感知的数据缓存策略。

## 1.2 国内外研究现状

缓解闪存单元的读干扰有很多不同的方式, 比如减弱读取时的电压、增强错误校验能力、改变数据分布方式、牺牲部分空间换取更大的读干扰容忍度等。以往的研究主要从修改固件行为、热读数据管理和缓存管理这三个方面着手解决读干扰问题, 因此, 接下来将着重从这三个方面来进行阐述。

### 1.2.1 通过修改固件行为缓解读干扰

许多研究人员试图通过固件级别的内部校正来解决读干扰问题。Kang 等人<sup>[12]</sup>通过在 NAND 单元串中增加一个额外的虚单元, 可以有效减少读操作对电压产生的读扰动。Zambelli 等人<sup>[13]</sup>分析了基于 TLC NAND 的闪存在均匀读干扰和集中读干扰两种条件下的不同行为, 阐述了不同工作负载模型对企业级闪存可靠性的影响。Li 等人<sup>[14]</sup>通过给不同闪存块设置不同的读取电压, 并不断对这些闪存块进行读取操作, 以此建立了一个模型来评估闪存块在不同读电压中的读干扰水平。Cai 等人<sup>[15]</sup>进行了一系列固件级别的实验, 表明读干扰效应对闪存擦除周期、数据保留时间、读响应时间等都有显著的不利影响。他们使用读重试技术<sup>[4,16]</sup>, 测量闪存块中所有闪存单元的初始阈值电压。然后, 从闪存块中选择要读取的单个页面, 对其执行 N 次重复读取操作, 以此来量化读干扰对电压分布的影响。提出可以利用 ECC 的纠错能力降低读电压水平, 这样的方法在大多数闪存单位都正常运转时可以很好的解决部分热读闪存块的读干扰问题。

NAND 闪存通常使用 ECC 校验模块来纠正每个页面中一定数量的原始比特错误。只要错误总数不超过 ECC 的校正能力, 就可以校正错误, 成功读取数据。当数据的保留时

间较短时, 闪存的整体原始比特误差率远低于保留时间较长的误差率, 这也导致了对于许多闪存块而言, 存在大量未使用的 ECC 校正能力<sup>[15]</sup>。当数据刚进入闪存时, 不存在保留错误或读干扰错误, 因为数据刚刚被恢复。在这些情况下, 强大的 ECC 校验能力允许设计一些读干扰缓解机制, 比如降低读取电压。尽管这样的机制会提高闪存的原始比特错误率, 但是不会超出 ECC 的校验能力范围, 而且可以有效缓解读干扰。当被读取的数据的原始误码率超出 ECC 校验能力时, 闪存并不会直接返回错误读取的结果, 而是通过改变电压再次进行读取, 称为读重试<sup>[17]</sup>。读重试操作不仅会延长读反应时间, 而且由于多次施加电压, 同样也有带来额外的读干扰。Park 等人<sup>[7]</sup>提出可以通过预设最佳读取电压, 将多个读重试操作缩短为一个最佳读操作, 实现更好的读取效果。

### 1.2.2 通过管理读热数据缓解读干扰

固件级别的修正方法对闪存的其他方面可能会有一些负面影响, 因此一些研究人员提出可以结合一些冷热数据管理技术来减少触发读重写操作的次数, 比如可以利用闪存控制器, 通过缓存最近的读数据来避免读操作。Ha 等人<sup>[18]</sup>则提出可以通过维护每个闪存块的累计读计数器, 并在计数器超过预定阈值时重写闪存块的内容。Kang 等人<sup>[19]</sup>将强化学习引入到读重写和垃圾回收调度中, 希望在不同的读写场景中找到执行数据移动和擦除操作的恰当时机。读干扰数据管理主要可以分为两个相互独立的步骤, 第一个是将热数据分散到多个不同的闪存块中, 第二个是在闪存块中调整冷热数据的位置。

Zhao 等人<sup>[20]</sup>提出了一种数据重分配方案, 指出可以将读重写闪存块中的数据分布到不同的闪存块上, 从而减少读重写操作和原始误码率。这样做的原理主要是希望通过综合考虑擦除周期和闪存块的读次数两个因素, 使每个闪存块可以达到自己的最大读取次数。Liu 等人<sup>[21]</sup>提出了一种去中心化的读干扰数据管理方法, 将读重写的回写数据分布到不同的闪存块中, 力求防止所有热读页面被回写到同一个闪存块中。但是这个方法没有考虑来自其他闪存块的热读页面, 所以热读页面聚集的可能性仍然很高。Ha 等人<sup>[18,22]</sup>开发了一种名为 redFTL 的读干扰管理技术, 该技术通过减弱读电压来提高部分闪存块的读干扰容忍性, 然后将读重写操作的回写数据写入到这些闪存块中。Wu 等人<sup>[23]</sup>提出的名为 IPR 的读干扰管理方法则是通过牺牲部分闪存块的空间来提高部分闪存块的读干扰容忍性。IPR 不将读重写产生的回写数据直接写入读干扰容忍性更高的闪存块中, 而是先将其写入一部分监视块中, 并计算每个页面的读取频率, 在第二次读重写时才将读计数相对较多的页面写入读干扰容忍性更高的闪存块中, 读计数相对较少的页面则写回到普通闪存块中。

以 TLC 闪存为例, 一个字线上有三个页面, 分别称为低页、中页、高页。读取低页只

需施加一次电压, 读取中页需要施加两次电压, 读取高页需要施加三次电压。相应的, 它们也会有不同的读写延迟和产生不同的读干扰。大多数相关工作<sup>[24-26]</sup>提出可以将读密集型数据放置到低页(更快的页面)上来提高读性能。然而, 闪存中的低页页面数量是有限的。如果所有读密集型的数据都分配给低页, 那么中页和高页(慢速页面)在这样的负载中就没有得到充分使用, 导致了极大的空间浪费。为了提高慢速页面的性能, Choi 等人<sup>[27]</sup>利用重编程技术使慢速页面可以获得与快速页面相同的低读延迟。重编程技术的原理是闪存单元中的电子可以持续增加而不能减少, 重编程技术将电压从低电平增加到高电平。因此, 一个闪存单元可以在被擦除前编程多次, 只要它的电压不断增加。Long 等人<sup>[28]</sup>提出了一种新型的冷热数据分类算法, 该算法通过感知数据失效时间来识别热只写数据、热只读数据、冷只写数据和冷只热数据。并且提出可以先将低延迟的低页和中页分配给热读数据, 已经分配到高页上的热读数据在恰当的时机重编程到低页或中页上, 以此来降低读干扰, 提高读性能。

### 1.2.3 通过缓存管理缓解读干扰

热数据管理的技术大多是在读重写发生之后对重写数据进行特殊放置, 避免第二次读重写发生。但是这些方法都对热数据的到来缺少应对措施, 即当读重写发生后才进行解决。直观上来讲, 如果可以用闪存系统中的 DRAM 缓存一部分热读数据, 则自然就可以减少闪存器件的读干扰。Shim 等人<sup>[29]</sup>提出可以根据工作负载的状态动态改变地址映射表和读写缓存的比例大小, 适当分区来达到更好的缓存目的。Hu 等人<sup>[30]</sup>提出可以利用闪存内部的芯片级和通道级并行性, 在预期流量爆发时明智和主动地清除脏缓冲数据, 在缓存命中率和响应时间方面实现更好的性能。Yeh 等人<sup>[31]</sup>则提出可以通过建立 NAND 闪存写期望最小值的统计模型, 计算出适合写缓冲区和写映射缓存的 DRAM 分配策略, 并采用成本效益策略, 将适当的 DRAM 空间分别从写缓冲区和写映射缓存重新分配给读缓冲区和读映射缓存, 从而在 NAND 闪存中减少更多的读/写, 提高响应时间。

在缓存替换策略方面, 除了传统的 LRU、LFU、ARC<sup>[32]</sup>和 LIRS<sup>[33]</sup>等策略之外, NAND 闪存中还存在一些基于闪存物理特性的缓存替换策略。Kim 等人<sup>[34]</sup>提出一种称为 BPLRU 的缓存替换策略, 他们提出可以以闪存块为单位, 位于同一闪存块中的数据在缓存中聚集在一起进行移动, 需要替换时同一闪存块中的数据一起移出缓存, 通过利用冷热数据聚集的特点在某些特定负载中实现更好的缓存效果。Wu 等人<sup>[35]</sup>提出一种称为 GCaR 的支持垃圾回收感知的替换策略, 其基本思想是在缓存时为处于 GC 状态的闪存芯片的闪存块提供更高的优先级, 通过这种方式可以在一定程度上减少闪存系统的写放大。Wu 等人<sup>[36]</sup>提出

还可以利用集群的空间局部性，构造一种具有粒度差异的混合页面/闪存块的缓存架构，实现更好的缓存效果。

### 1.3 研究主要内容

本论文主要研究读密集场景下，基于 NAND 闪存物理特性的读干扰缓解策略中的相关理论和方法。设计方案最终效果受数据访问行为、数据缓存策略、数据放置策略、闪存垃圾回收机制等多个因素的影响。数据访问行为（例如，数据访问的频度及热漂移）主要由上层应用决定，而数据缓存策略和数据放置策略则可以对其进行设计和优化从而延长闪存寿命或缩短时延。本文提出了一种基于访问行为的数据放置策略 Cocktail 和一种基于读干扰感知的数据缓存策略 Minato，用来提高受到读干扰影响的闪存系统的各项性能指标。研究的主要工作如下：

(1) 在基于访问行为的数据放置策略 Cocktail 中，首先分析了传统写入模式下闪存块中的热度分布特点和规律，明确闪存由于数据访问行为而形成热区的形成机理，为利用数据放置来减少闪存块中的热聚集并提高闪存读取效率打下基础；其次通过研究分析出闪存物理特性和闪存可靠性之间的内在联系，探明不同物理位置的读取热度对闪存块电压分布的影响，为设计和优化数据放置策略提供理论支持；之后研究并分析了闪存缓存机制对数据放置策略的影响，揭示数据访问行为、闪存缓存机制、数据放置策略之间的内在规律，重新审视并设计了基于热读场景下的闪存数据放置策略；最后，通过观察分析出了适合以块粒度或者以页粒度进行数据放置的不同应用场景，揭示不同放置方式可能存在的利弊关系，研究并设计了一种适用于多场景的基于闪存物理特性减少闪存块读干扰的自适应数据放置策略。

(2) 在基于读干扰感知的数据缓存策略 Minato 中，首先研究并分析了不同应用场景下的数据访问行为的内在规律，构造合适的数学描述模型，建立数据访问行为的提取、表达、和评估机制，以便高效地跟踪和预测不同应用场景下的数据访问行为，从而为设计和优化缓存策略奠定基础；之后分析了闪存中的缓存机制对闪存读写的影响，探明在缓存空间中提取数据信息的可能性，预测未来一段时间窗口内的数据冷热属性；最后通过衡量基于闪存物理特性和数据访问行为规划不同缓存区域对缓存命中率的影响，分析根据数据不同写入方式将数据按照不同粒度进行聚集的时效性和合理性，提出了一种可以按区域特征来吸纳数据的缓存策略。



## 1.4 论文章节安排

本论文总共分为五个章节，内容具体安排如下：

第一章：绪论，介绍了本论文的研究背景及意义，并结合相关资料讨论了该研究方向目前的国内外研究现状，随后概括了本论文研究的主要内容。

第二章：NAND 闪存存储器概述，介绍了 NAND 闪存的结构及工作原理，阐述了闪存转换层 FTL 的存在原因、组织架构和主要功能，总结了闪存可靠性方面的相关工作。

第三章：基于访问行为的数据放置策略 Cocktail，首先从研究动机入手，指出闪存中会因为数据访问行为而形成热区，可以利用数据放置来减少闪存块中的热聚集，表明 Cocktail 的合理性，其次详细介绍了 Cocktail 策略的设计方案与实现细节，最后通过实验验证了 Cocktail 相比于其它数据放置策略有更好的性能表现。

第四章：基于读干扰感知的数据缓存策略 Minato，首先分析闪存中的缓存机制和地址映射表与闪存读写请求的关系，通过衡量基于闪存物理特性和数据访问行为规划不同缓存区域对缓存命中率的影响，分析 Minato 的合理性，其次详细介绍了 Minato 策略的设计方案与实现细节，最后通过实验验证了 Minato 相比于其它数据缓存策略有更好的性能表现。

第五章：总结了本论文中已完成的工作，分析现有工作存在的不足并提出未来可行的优化方向。

## 2 NAND 闪存存储器概述

NAND 闪存作为数据存储的主要方法被广泛应用于计算机系统。与传统磁盘相比, NAND 闪存固态硬盘拥有更显著的读写性能, 在随机访问输入/输出 (I/O) 操作方面有数量级的提升, 并且更耐冲击、具有更小的外形尺寸和更少的能耗。图 2.1 中展示了现代闪存系统的整体结构。闪存系统首先通过主机接口与外部系统进行对接并接收读写请求, 在闪存控制器中通过多个模块如缓存管理模块、地址转化模块、可靠性管理模块、数据校验模块等共同配合管理数据, 数据最终会写入到闪存芯片中去。在接下来的章节中将详细介绍闪存芯片的组织结构及其工作原理、闪存控制器的主要模块、闪存读干扰的原因和相关应对机制。

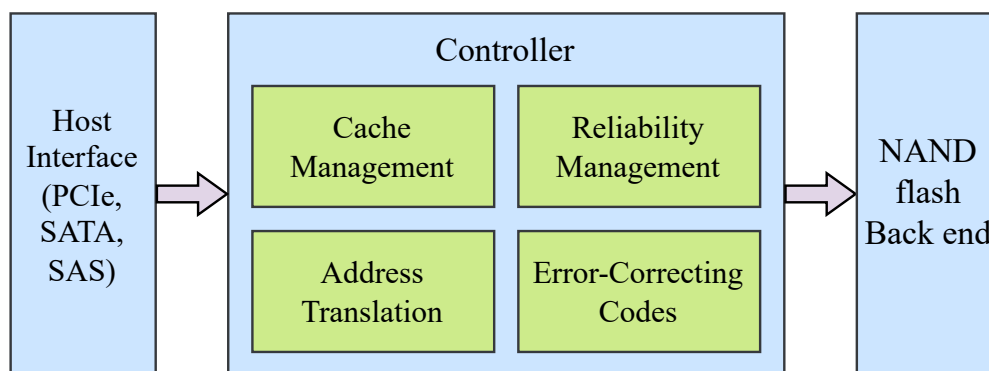


图 2.1 闪存的内部结构示意图

### 2.1 NAND 闪存结构及工作原理

#### 2.1.1 NAND 闪存芯片的组织结构

图 2.2 展示了现代 NAND 闪存物理介质的组织结构。最上层的结构是通道 (channel), 这是 NAND 闪存中最大的并行单位, 多个进程在做性能隔离时可以选择为每个进程的请求匹配一个通道; 每一个通道下面一般是一个芯片 (chip), 每一个芯片有一套完整的外围电路和外部信号线; 芯片下面又通过多层堆叠技术叠放了多个颗粒 (die), 颗粒之间共享外部信号线, 但是都有自己独立的内部信号线, 用于查询当前的状态; 颗粒由多个闪存片 (plane) 组成, 闪存片是 NAND 闪存中最小的并行单位, 且 Copyback 命令也是在此层面上进行 (指擦除时将有效页面迁移到同闪存片的其它空闲位置); 闪存片往下是多个闪存块 (block), 擦除操作的执行单位就是一个闪存块; 闪存块中由一根根字线 (WL) 和位

线 (BL) 纵横交叉组成, 每一根字线上可以形成多个页面, 闪存类型不同页面数量也有所差异, 以 TLC 为例, 一根字线上具有三个页面, 分别是高页 (MSB)、中页 (CSB)、低页 (LSB)。

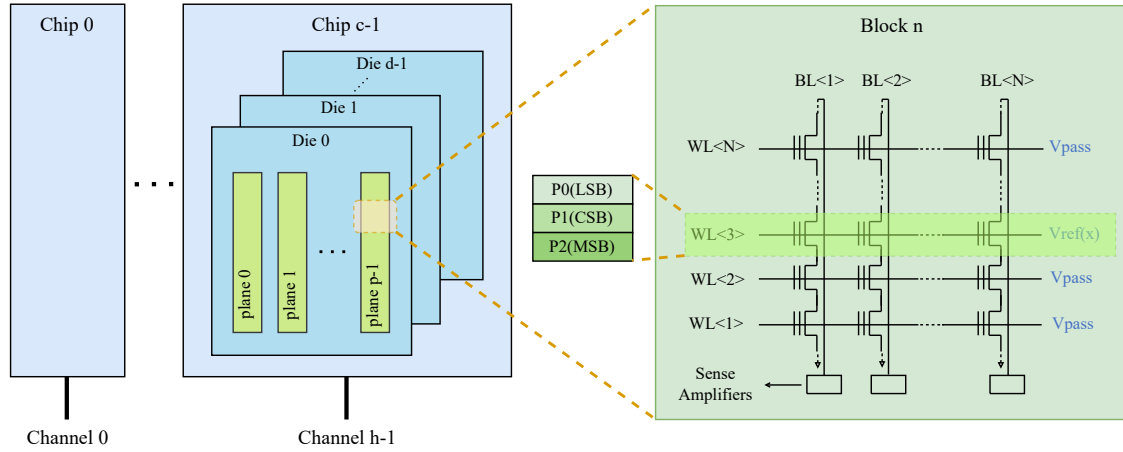


图 2.2 闪存块的物理结构

### 2.1.2 NAND 闪存的数据存储原理

NAND 闪存将数据存储为每个闪存单元的阈值电压, 该闪存单元由浮栅晶体管组成。对于单级单元 (SLC) NAND 闪存而言, 每个闪存单元可以存储一个 1 比特位的值, 并可以编程为两个阈值电压状态之一, 称之为 ER 和 P1 状态。多层单元 (MLC) NAND 闪存在每个单元中存储一个 2 比特位的值, 有 4 种可能的状态 (ER, P1, P2, P3), 三层单元 (TLC) NAND 闪存在每个单元中存储一个 3 比特位的值, 有 8 种可能的状态 (ER, P1 - p7)。每个状态代表一个不同的值, 并在所有可能的阈值电压范围内分配一个电压窗口。由于程序操作的变化, 编程到相同状态的闪存单元的阈值电压最初分布在这个电压窗口中。

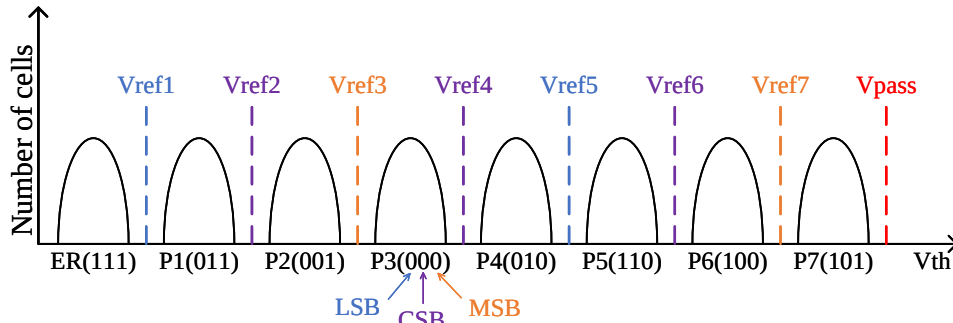


图 2.3 NAND 闪存的电压分布

图 2.3 为 TLC NAND 闪存的阈值电压分布。x 轴表示阈值电压, 它跨越一定的电压范围。y 轴显示了所有闪存单元上每个电压水平的概率密度。每个阈值电压状态的阈值电压

分布可以表示为跨越状态电压窗口的概率密度曲线。图中用状态的名称和相应的位值标记每个状态的分布曲线，相邻状态的位值由 1 的汉明距离隔开。TLC 的比特位值可以分解为 MSB、CSB 和 LSB。相邻阈值电压窗口之间的边界，在图 2.3 中的 TLC 电压分布中标记为  $V_{ref1}$ 、 $V_{ref2}$ 、 $V_{ref3}$ 、 $V_{ref4}$ 、 $V_{ref5}$ 、 $V_{ref6}$ 、 $V_{ref7}$ ，称为读参考电压；闪存控制器可以使用这些电压值在读取闪存单元时识别每个单元的电压状态。

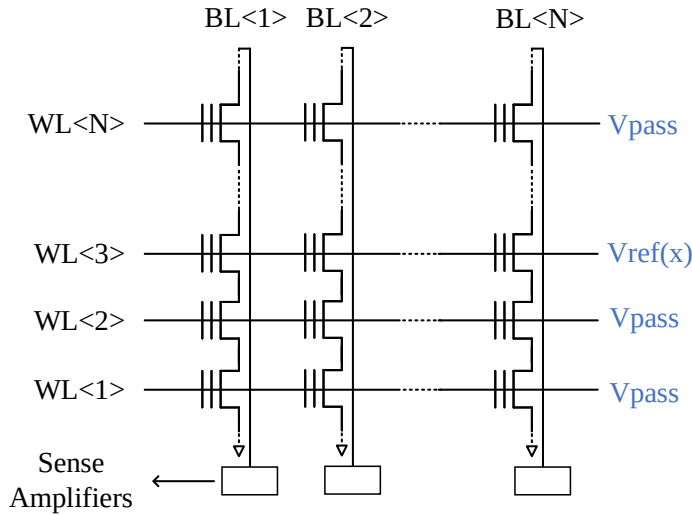


图 2.4 NAND 闪存的块结构

图 2.4 为 NAND 闪存块的内部组织结构。每个闪存块包含多行闪存单元（通常是 128-512 行）。每行闪存单元由一条共同的字线（WL）连接在一起，通常跨越 32K-64K 个闪存单元。沿着字线的所有闪存单元在 SLC NAND 闪存中形成一个页面；对于 MLC NAND 闪存而言，同一字线上所有闪存单元的 MSB 组合成 MSB 页，同一字线上所有闪存单元的 LSB 组合成 LSB 页；类似地，TLC NAND 闪存中将每个字线上所有闪存单元的 MSB 组合成 MSB 页、CSB 组合成 CSB 页、LSB 组合成 LSB 页。在 MLC NAND 闪存中，每个闪存块包含 256-1024 个闪存页面，每个页面的大小通常为 8-16kB。

### 2.1.3 NAND 闪存的读取操作

通过在每个单元的控制门上施加读参考电压，可以感知每个单元的阈值电压，从而在 NAND 闪存中读取数据。要读取存储在 SLC NAND 单元中的值，只需要区分位值为 1 的状态和位值为 0 的状态，因此只需要使用一个读参考电压。同样地，假如要读取 MLC NAND 单元中的 LSB，只需要区分 LSB 值为 1 的状态（ER 和 P1）和 LSB 值为 0 的状态（P2 和 P3），这同样可以只通过单个读参考电压来实现。而如果要在 MLC NAND 中读取 MSB 页面，则需要区分 MSB 值为 1（ER 和 P3）的状态和 MSB 值为 0（P1 和 P2）的状态。因此，

需要分别施加两个读参考电压（这需要两次连续的读取操作）来确定 MSB。从 TLC NAND 闪存单元中读取数据类似于 MLC NAND 闪存单元的数据读取过程，即读取 TLC NAND 的 LSB 只需要施加一个读参考电压，读取 CSB 需要施加两个读参考电压，读取 MSB 需要施加三个读参考电压。

如图 2.4 所示，来自多个字线（图中的 WL）的单元在共享位线（BL）上串联连接到感应放大器（Sense Amplifiers）。该放大器将从块中读取的值传输到 plane 的内存通道中。为了从位线上读取到单个单元，必须打开同一位线上的所有其他单元（即未读单元），以允许正在读取的值通过串联通道到达感应放大器的位置。NAND 闪存需要通过在未读单元的字线上施加直通电压（V<sub>pass</sub>）来实现这一点。当直通电压（即最大的阈值电压）施加到闪存单元时，无论浮栅电压如何，电池晶体管的源极和漏极都连接起来，则所有未读单元都被打开，读电压便可顺利到达感应放大器。

#### 2.1.4 NAND 闪存的写入和擦除操作

当在进行写入操作时，对控制门施加高正电压后，电子会从衬底注入闪存单元的浮栅。与读取操作类似，NAND 闪存会在欲读取的字线上施加一个写电压，同时，在未读单元的字线上施加直通电压。当数据被写入时，电荷通过一种称为增量步进脉冲编程（increment step pulse programming, ISPP）<sup>[37]</sup>的方式将电子从 FN 隧道传输到浮栅。ISPP 编程方式其中一个目的，就是控制闪存单元电压值，将其控制在一个预定电压范围来提高闪存单元写入操作的可靠性。在现代 NAND 闪存中，单个字线的所有闪存单元都是同时编程的。在编程过程中，当字线上的一个单元达到其目标电压而其他单元尚未达到其目标电压时，ISPP 通过关闭单元位线的 SSL 晶体管来抑制向该单元发送编程脉冲。在 SLC NAND 闪存中，使用一次性写入，其中对单元进行写入所需的所有 ISPP 步进脉冲都被应用，直到字线的所有单元都被完全写入。一次性写入不会将一个字线的写入操作与另一个字线的写入操作交织在一起。而在较新的 MLC NAND 闪存中，写入操作之间缺乏交织，通常会对相邻字线的单元引入大量的程序干扰。在 TLC NAND 闪存中，采用了一种称为 foggy-fine programming<sup>[38]</sup>的编程方法来减轻这种干扰。闪光单元首先根据其 LSB 值部分编程，使用二进制编程步骤，其中使用较大的 ISPP 步进脉冲来显著提高电压水平。然后，基于它的 CSB 和 MSB 值再次对闪存单元进行部分编程，以获得一组新的临时状态。

在 NAND 闪存中，只有擦除闪存单元中的现有数据后，才能用新数据重新编程。这是因为 ISPP 只能增加电池的电压，擦除操作可以将闪存块中所有单元的阈值电压状态重置为 ER 状态。在擦除操作期间，通过在电池晶体管上设置一个高负电压，诱使电子从闪存

单元的 FG 回到衬底。又因为闪存块中的所有单元共享一个共同的晶体管基板（即，闪存块中的所有晶体管主体连接在一起），擦除操作会使得闪存块中的所有数据都被擦除。

## 2.2 闪存控制器

### 2.2.1 闪存转换层 FTL

闪存转换层（Flash Translation Layer, FTL）运行在 NAND 闪存系统内部的微处理器上，主要的任务是处理主机端下发的请求以及管理缓存。对于使用 NVMe 协议的 NAND 闪存来说，在 FTL 中处理 I/O 请求需要四个步骤。首先，当 Host-Interface Logic (HIL) 从请求队列中选择一个请求时，它会将该请求插入到一个设备级队列中。其次，HIL 将请求分解为多个闪存事务，其中每个事务都是单个页面的粒度。接下来，FTL 会检查该请求是否是写请求。如果是并且闪存系统支持写缓存的话，则写缓存管理单元将每个事务的数据存储在 DRAM 内的写缓存空间中，并要求 HIL 准备响应。否则，FTL 将事务的逻辑页地址（LPA）转换为物理页地址（PPA），并将事务编入相应的芯片级队列，即读队列（RDQ）或者写队列（WRQ）。事务调度单元（TSU）会解决芯片级队列中待处理事务之间的资源争用问题，并将可执行的事务发送到相应的 FCC<sup>[39-40]</sup>。最后，当一个请求的所有事务完成时，FTL 要求 HIL 进行响应，然后将其交付给主机。

FTL 的地址转换模块（Address Translation Module）在实现异地更新中起着关键作用。当接收到事务的 LPA 时，页面分配机制将 LPA 分配给一个空闲的 PPA。逻辑页地址到物理页地址的映射记录在映射表中，映射表存储在非易失性内存里，并缓存在 DRAM（以减少映射查找的延迟）中<sup>[41]</sup>。当事务读取 LPA 时，地址转换模块会搜索 LPA 的映射并检索到 PPA。

FTL 还具有磨损均衡机制（WL）和垃圾回收机制（GC）<sup>[42]</sup>。当空闲页面数量低于阈值时 FTL 会触发 GC。GC 过程中会选择一个具有大量无效页的候选块，并回收闪存块的无效页，将闪存块中的所有有效页移动到一个空闲块中，然后对候选块进行擦除。任何在擦除期间到来的读写请求都会被插入到专用的读队列（GC-rdq）和写队列（GC-wrq）中，因此事务调度单元在空闲期间也可以调度与 GC 相关的请求。

### 2.2.2 闪存可靠性管理

NAND 闪存控制器会执行许多后台优化，从而提高闪存的可靠性。这些闪存可靠性管理技术可以以非常低的成本有效地提高闪存的寿命。而且这些优化通常在空闲时间执行，可以最小化对运行工作负载的干扰。这些管理技术有时需要在内存中存储较小的元数

据（例如，用于存储最佳读参考电压<sup>[15,43-44]</sup>），或需要一个计时器（例如，用于及时触发刷新<sup>[45]</sup>）。闪存可靠性管理是现代闪存系统保证自身可靠性的重要手段。由于知道在相邻闪存单元中编程的值可以更容易地正确确定存储在当前闪存单元格中正在读取的值<sup>[46]</sup>，因此可以利用相邻闪存单元的值辅助纠错。闪存的读重试（Read Retry）<sup>[47]</sup>可能需要多次重试步骤才能正确获取数据，通过可靠性管理模块保存最优的读参考电压可能可以大大缩短闪存读取时间<sup>[7]</sup>。

闪存可靠性管理另一个重要模块是坏块管理。由于工艺变化或磨损不均匀，少数闪存块可能具有比一般闪存块高得多的原始误码率（RBER）。减轻或容忍这些闪存块上的 RBER 通常需要很高的成本。因此，更有效地将这些闪存块识别并记录为坏块，避免使用它们存储有用的数据通常更能提高闪存的可靠性。这种方式称为坏块管理。一般有两种类型的坏块：原始坏块（OBB）和增长坏块（GBB），原始坏块是由于制造问题（例如，工艺变化）而导致的缺陷，增长坏块则是在闪存运行过程中由于磨损等原因失效的。闪存供应商在制造闪存芯片时就会执行广泛的测试，称为坏块扫描，以识别 OBB。最初，所有块都保持在已擦除状态，并且每个字节都包含值 0xFF。在每个 OBB 中，坏块扫描过程将一个特定的数据值（例如 0x00）写入块内的特定字节位置，该位置表示块状态。一个没有缺陷的闪存块不会被修改，因此它的块状态字节保持在值 0xFF。当闪存设备第一次上电时，闪存控制器会遍历所有闪存块，并检查每个块状态字节中存储的值。任何不包含 0xFF 值的块都被标记为坏块，并记录在存储在控制器中的坏块表中。每个平面中有少量块被预留为预留块（即正常操作中不使用的块），坏块表会自动将原本指向 OBB 的操作重新映射到其中一个预留块。坏块表将 OBB 重新映射到同一 plane 中的预留块，以保证闪存系统在写入数据时可以保持相同的并行度，从而避免性能损失。正常来说闪存系统中只有不到 2% 的块是 OBB。

### 2.2.3 闪存校验机制

Error-Correcting Codes (ECC) 是用来检测和纠正在闪存中出现的原始比特错误的校验机制。当主机写入一页数据后，闪存控制器会将其分割成一个或多个部分。对于每个部分，控制器会为其生成一个码字，由该数据和纠正码组成。ECC 提供的保护强度由编码率决定，编码率是数据大小除以码字大小。较高的编码速率提供较弱的保护，但消耗的存储空间更少，这是闪存系统中可靠性权衡的一个关键点。

采用的 ECC 算法（一般为 BCH<sup>[48-49]</sup>或 LDPC<sup>[50-51]</sup>）以及码字的长度和编码率，决定了总纠错能力，即 ECC 可以纠错的最大原始误码数。除了 ECC 信息外，码字还包含循环

冗余校验和 (CRC) 校验信息。当从 NAND 闪存读取数据时, 有时 ECC 算法可能错误地指示它已成功更正了数据中的所有错误, 但其实未更正的错误仍然存在。为确保不正确的数据不返回给用户, 控制器在硬件中执行 CRC 检查, 以验证数据是否是正常的<sup>[52]</sup>。除了 ECC 来防止比特级错误, 许多闪存系统还使用类似 RAID 的奇偶校验<sup>[53-54]</sup>。它的关键点在于给每个超级页面存储奇偶校验信息, 以保护数据不受单个 chip 或 plane 发生的 ECC 故障的影响。

## 2.3 NAND 闪存可靠性分析

### 2.3.1 闪存数据出错的原因

随着闪存制造商不断提高 NAND 闪存的密度, 为了减小闪存单元的尺寸只能使用更激进的制造工艺技术。因此, 每个储存电荷的闪存单元容量都变小了, 闪存单元之间的距离也更近了。这些变化降低了 NAND 闪存的可靠性, 从而增加了新一代 NAND 闪存中读取错误的概率。当单元阈值电压 ( $V_{th}$ ) 无意中发生变化时就会改变控制器观察到的单元状态, 从而导致读取错误的发生。误差可能由一系列来源引起, 主要可将其分为四类: 工艺变化误差、数据保留误差、写入引起的误差和读取引起的误差。

工艺变化误差是指在制造过程中发生的误差变化。比如, 在单个闪存芯片中, 由于现代制造工艺技术的限制, 不同的闪存单元可能会具有不同的属性<sup>[47,55]</sup>。因此, 闪存单元之间存在固有的变异, 有些闪存单元比其他闪存单元有更高的错误率。

数据保留误差<sup>[43]</sup>是另一种常见的错误类型。在对闪存单元进行编程后, 由于电荷随着时间的推移从晶体管中漏出, 数据保留误差会随着时间的推移而增加和积累。当电荷从闪存单元中泄漏时, 闪存单元的阈值电压 ( $V_{th}$ ) 也会随之降低。在 NAND 闪存中, 如果不采用积极的刷新技术<sup>[45]</sup>, 保留误差会越来越严重并成为所有闪存错误的主要来源<sup>[47]</sup>。

写入引起的误差发生在写入或擦除操作期间。写入引起的误差<sup>[4,44,56]</sup>是在擦除和写入闪存页面后立即发生的错误。这些错误的发生是因为写入和擦除操作具有不准确性, 这种不准确性导致了一些闪存单元被编程成另一种状态, 而不是它所期望的目标状态。在闪存单元的使用寿命中, 随着更多的擦除操作发生会导致更多的电子被困在晶体管, 这被称为磨损。磨损增加了编程和擦除操作期间的不准确性, 从而增加了写入引起的误差。单元和单元之间的干扰错误是另一种类型的写入引起的误差, 当一个字线的相邻单元被编程时, 它会增加该字线单元的阈值电压。由于相邻单元之间存在寄生电容耦合, 当对一个单元施加高编程电压时, 电容耦合会给相邻单元的晶体管增加电荷, 从而增加程序干扰误差。



读取引起的误差<sup>[57-59]</sup>是一种由于频繁读取而引起的错误，这会导致对闪存单元的两次读取可能返回不同的数据值。这种类型的错误是由于频繁对未读单元施加直通电压 ( $V_{pass}$ ) 而发生的。当字线上的一个单元被读取时，对未读单元应用  $V_{pass}$  会对未读单元产生微弱的编程效应，将电子缓慢地转移到未读单元的晶体管中，并增加了未读单元的阈值电压，从而导致误差发生。

### 2.3.2 读干扰的相关机制

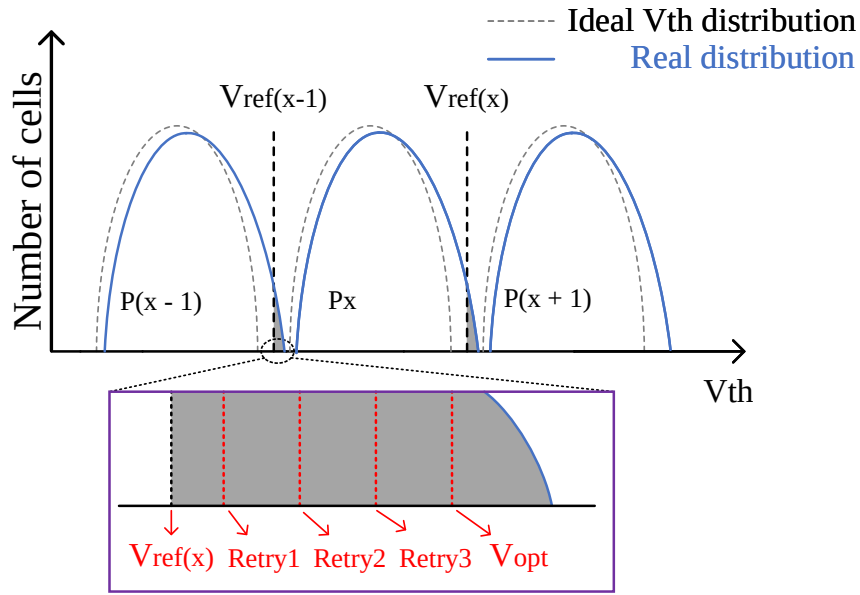


图 2.5 读干扰的电压变化和读重试操作

如图 2.5 所示，严重读干扰会使得电压分布整体向右偏移，甚至超出原先设定的读参考电压范围。由章节 2.1.3 的描述可以得知，闪存是根据施加读参考电压后电压的表现状态来界定数值的。当电压分布发生偏移之后意味着会出现相当程度的读取错误。为了保证存储数据的可靠性，现代闪存系统中普遍采用纠错码 (error-correcting codes, ECC)。ECC 可以通过在码字中存储冗余位 (即 ECC 奇偶校验) 来检测和纠正数据单元中的错误码。当严重读干扰导致的数据读取错误超出 ECC 校验机制的最大校验能力时，读取操作就会失败。

读重试操作<sup>[4,17,47,60]</sup>是为了解决读取失败问题而设定的机制。图 2.5 展示了读重试操作的基本过程。如图所示，当读参考电压为  $V_{ref}(x)$  时，读取错误的比特数超出了 ECC 校验机制的校验能力。因此，在读取失败之后触发读重试机制，读重试机制通过预先设置的一个差值等量的调整读参考电压，重新进行读操作尝试，如图中的 Retry1、Retry2、Retry3 步骤。最后，在多次尝试之后通过一个最优的电压  $V_{opt}$  成功读取数据。在这个过程中，读取一个页面数据经常需要多次读取尝试，严重影响读反应时间。

当某个闪存块受到严重读干扰从而影响到数据可靠性时，闪存系统中还存在另外一个缓解读干扰的机制，称为读重写（read reclaim）。在许多现代闪存中，在只有读请求的场景下也会发生多次擦除操作<sup>[11]</sup>。读重写的关键思想是在一个闪存块遭受严重读干扰问题时对其进行擦除，并将其中的有效数据重写到另一个新的闪存块中。当一个闪存块的读次数超过预设的阈值即可容忍的最大读次数（产生严重读干扰）时会触发读重写。由于读重写操作需要将被干扰的闪存块数据重新写到新的闪存块中，因此会产生大量的回写数据，这对 NAND 闪存的响应时间影响很大。同时由于 NAND 闪存的擦除操作有一定的次数限制，因此频繁触发读重写操作也会缩短 NAND 闪存的寿命。

## 2.4 本章小结

这一章节首先介绍了 NAND 闪存存储器的组织结构、数据存储原理和读写擦除操作的工作原理。接着介绍了闪存控制器中的闪存转换层、闪存可靠性管理和闪存数据校验机制。最后分析了 NAND 闪存数据出错的原因以及读干扰的相关机制。

### 3 基于访问行为的数据放置策略 Cocktail

在基于 NAND 闪存的固态硬盘中,为了缓解读干扰问题,系统会在后台执行大量读干扰引起的读重写操作(擦除严重读干扰的闪存块并将有效页面重写到其它位置)。在读密集型工作环境下,读重写操作会显著降低固态硬盘的性能,缩短固态硬盘的使用寿命。为了解决这个问题,本文提出了一种新的读干扰管理方法,称为 Cocktail。它混合了一小部分热读页面和大比例的冷读页面,从而避免将热读页面聚集到几个块中。由于了解到读重写操作经常是由热读页面的聚集而触发的,因此, Cocktail 首先将从用户请求中提取的冷读数据预先装填进写入池中的闪存块,但是不会完全填满,而是在闪存块中预留了一部分空间。然后, Cocktail 用读重写引起的回写数据来填满这些预填充了冷数据的闪存块,创建读平衡块。通过将写入池容量和读重写数据与用户写入数据的比例这两个阈值集成到 Cocktail 中,可以控制读重写引起的写回数据与块中用户请求数据的比值。Cocktail 会根据数据特征动态调整这两个阈值。Cocktail 有利于将读重写引起的热回写数据分散到更大的闪存块范围,从而减少第二次读重写的发生以及平均闪存块的读次数。本章节将 Cocktail 与三种现有的方案 Double write frontier、redFTL 和 IPR 进行了比较,分别在十种真实工作负载条件下测量了四种方案的各项指标。实验结果表明,与现有方案相比, Cocktail 能有效提升固态硬盘性能。

#### 3.1 引言

3D NAND 闪存是目前广泛使用的存储设备。读干扰噪声已成为高密度 3D NAND 闪存中待解决的主要可靠性问题。传统的读干扰管理技术已经不足以缓解读干扰问题。为了解决这个问题, Cocktail 的重点放在了减少热读聚集型闪存块的数量和减少由读重写引起的回写数据的数据量上。Cocktail 是一种具有普适性的读干扰管理方法,它利用一种分散热点的分布策略,将读重写的热回写数据放置到大范围的闪存块上。这种分散的分布策略有利于将对数据的读访问平衡到多个闪存块中,从而减少热读闪存块的数量和读重写操作的发生。Cocktail 的核心是写边界块和写入池,它们将冷数据和热数据协同组合成读取平衡的闪存块。值得注意的是,冷热数据是根据读取频率来区分的。不同的工作负载有不同的数据访问行为,没有一个统一的热度标准,因此数据的冷热属性是相对而言的,比如一个数据比另一个数据有更高的读取频率,那么相对来说该数据就是较热的数据。在 3.3 章

节中将会阐述写边界块如何从用户请求中提取冷数据并将其预先放入闪存块中。Cocktail 写入池的作用主要在于将读重写引起的回写热数据填充到预填充了用户数据的闪存块中，从而创建出读平衡的闪存块。

Cocktail 提出的三个动机具体如下：

(1) 在日渐普遍的热读场景中，基于 3D NAND 闪存的固态硬盘设备迫切需要解决严重读干扰问题。

(2) 固态硬盘为了减少读干扰而进行的读重写操作会显著降低硬盘的读写性能。

(3) 为解决读干扰引起的问题而设计的读干扰管理技术已经不适用于现代固态硬盘中。

读干扰问题产生的主要原因是闪存块读请求的累积。当读取闪存块中的一个页面时，NAND 闪存必须对同一闪存块中的其它页面施加一个直通电压，这就好比一个弱编程操作。这种操作在闪存块的读取次数过多后，会严重扰乱闪存块的电压分布。NAND 闪存必须通过读重试操作（改变读取电压后再次读取），重复读取同一页面，直到被扰乱的页面数据被 ECC 机制正确校验<sup>[7]</sup>。因此，当发生读干扰<sup>[61]</sup>时，NAND 闪存读取页面所需的时间将延长数倍。更糟糕的情况是，当对一个受干扰的页面进行多次读取后仍然没有获得正确的数据时，NAND 闪存将永久丢失受干扰的页面中的数据，严重影响闪存可靠性。

读重写操作（RR）对现代固态硬盘<sup>[11]</sup>的响应时间和服务寿命有很大影响。读重写是 3D NAND 闪存中缓解读干扰问题的常用方法<sup>[11]</sup>。与垃圾回收操作（GC）<sup>[9]</sup>类似，如果受干扰的闪存块中存在有效数据，则必须将有效数据写入新的空闲闪存块中（即读重写操作引起的回写数据），然后对受干扰的闪存块执行擦除操作。一个电压被扰动的闪存块可以在被擦除后完全恢复。在众多读密集型工作负载下，闪存系统必须触发大量的擦除操作来消除可能产生的读错误。这些擦除操作将延长响应时间，增加写放大，并缩短 NAND 闪存的使用寿命。特别是，如果正常的 I/O 请求与 RR 操作发生冲突，则正常的 I/O 请求必须让步，并且可能会延迟很长时间才能得到服务。这种响应时间的延迟会严重降低 I/O 密集型应用程序的服务质量<sup>[62-63]</sup>。此外，由于闪存块的擦除次数会有一个限制，超出擦除限制的闪存块就不再可用，因此读重写中触发的不必要的擦除操作会显著缩短 NAND 闪存的寿命。

为了解决读重写引起的问题，现有的研究已经提出了许多读干扰管理方法<sup>[18,21,23]</sup>。许多现有的方法共享一个简单的策略：在读重写操作发生后，将受干扰闪存块的数据写回到同一闪存块。不幸的是，当使用这个简单的策略时，热读数据仍然会聚集到一起，继续扰动新写入的闪存块，这将触发第二次的读重写操作。与这种简单的回写策略相反，其他

一些研究将读重写数据分布在不同的页面上，旨在避免将所有热读数据集中到少数闪存块<sup>[21]</sup>中。然而，由于这种简单的去中心化策略没有意识到数据的访问行为的影响，热点数据很可能再次聚合成热点块，从而触发第二次读重写操作。与之前的读干扰管理解决方案不同，Cocktail 的分散策略根据数据的访问行为动态调整闪存块中热点数据的比例。通过观察（见章节 3.2）表明，大多数由读重写操作引起的回写数据是热读数据。因此，从用户写请求中提取冷数据，提前按比例放入一个闪存块中，并将块的剩余空闲部分空间预留给未来的热回写数据填充是可行的。遵循这些设计原则，当 RR 回写数据到达时，Cocktail 利用写入池将热回写数据快速组合到预填充了冷数据的闪存块中，从而防止第二次读重写并创建读平衡块。

综上所述，Cocktail 主要为提高 NAND 闪存的可靠性和性能做出了以下三点贡献：

- (1) 分析了读干扰错误的根本原因以及读重写对 NAND 闪存的不利影响，这有助于查明以前的读干扰管理方法的局限性和缺点。
- (2) 为了提高高密度 NAND 闪存的可靠性和性能，提出了一种低成本、高效率的读干扰管理解决方案 Cocktail。Cocktail 应用了一种分散的回写策略来平衡对闪存块的读访问，从而减少了读重写的次数和闪存块的读次数。
- (3) 用十个真实环境中的工作负载验证了 Cocktail。实验结果表明，Cocktail 在擦除操作的次数、读响应时间和写放大方面远远优于其他三种管理方法。

## 3.2 NAND 闪存数据访问行为分析

在提出有效可行的数据管理策略之前，本文首先需要探清数据的访问行为。在真实环境中，从用户往上层软件中读写数据到数据在闪存系统中落盘，数据在中间往往会被进行多次规划或者重定向，比如数据可能经过多个不同的节点，遇到一层或者多层缓存机制等。实际系统中进入闪存的数据和应用程序读写数据的数据访问行为可能会完全不同。因此，需要对真实环境中闪存系统的数据访问行为进行识别和预测，为缓解读干扰的数据放置策略提供数据预判信息。

为了理解真实环境中负载的数据访问行为，实验部分获取到微软、亚马逊、阿里巴巴等公司发布的真实环境下闪存系统工作负载。在相同实验条件（详见 3.4 章节的实验设置）下测试了十个工作负载的数据特征。图 3.1 展示了不同工作负载中相同请求的访问频率和数据访问行为。具有相同 Request\_id 的请求具有相同的逻辑地址和请求大小。Frequency 表示具有相同 Request\_id 的请求的出现次数。有一些请求的频率超过了 y 轴的最大值，这些

请求以最大值进行表示，以使层次结构在图中更明显。以最大值为标准，用蓝色标记那些频率小于最大值一半的请求，用红色标记那些频率大于最大值一半的请求。从图 3.1 中可以看出，在不同的工作负载中，读取频率相对较低的数据要多于读取频率相对较高的数据。

通过在 ali206 这一负载上进行的另一组实验，能够进一步研究真实工作负载的特征。ali206 包含了超过 28 亿个读请求，实验部分在模拟器平台上完整运行了全部请求，并在闪存系统最后一次发生读重写时记录了该闪存块所在 plane 上的其他闪存块的读计数（总共触发了 28507 次读重写操作）。图 3.2 绘制了实验结果，其中只显示了完整填充数据的闪存块的读计数状态。在图 3.2 中，每一个小方格代表一个闪存块，小方格的颜色代表了当前的读干扰程度（即当前的读计数）。与在图 3.1 中的方法类似，图 3.2 中闪存块读计数太大的数值用一个最大读计数表示，该读计数值设置为 10000。从实验结果可以观察到，这个 plane 中只有少数块受到严重的读干扰，而大多数块几乎没有遭受读干扰。

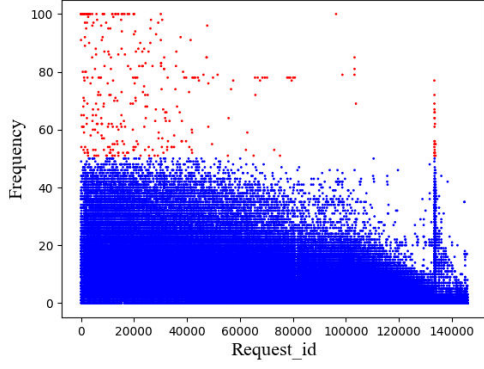
这些观察表明，高访问频率的数据往往倾向于聚集在一起，从而导致了严重的读取干扰。受到这些引人注目的发现的启发，可以判断存在一种数据管理策略，能够将少量热数据按比例与大量冷数据混合，并且将读取干扰的压力分布到不同的闪存块上。进一步的，Cocktail 这一数据管理策略背后还有另一个设计原则，即闪存系统中造成擦除的原因应该是垃圾回收操作，而不应该是读重写操作。更具体地说，只要可以将闪存块的读干扰水平控制在较低级别，就可以避免在触发垃圾回收机制之前提前去触发读重写机制。值得注意的是，当闪存块由于垃圾回收而被擦除时，闪存块中的数据也可以从读干扰中恢复过来。Cocktail 的实现细节将具体在 3.3 章节中阐述。

### 3.3 Cocktail 的设计与实现

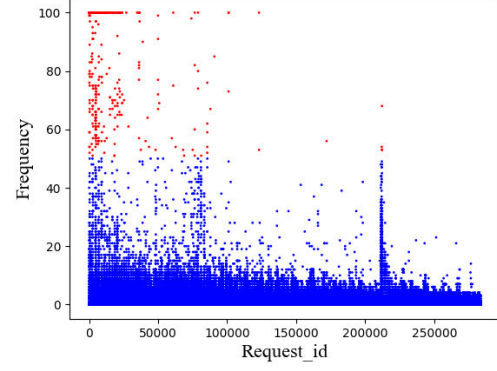
#### 3.3.1 读重写数据的回写原则

本章节介绍 Cocktail 中关于读重写数据回写时遵循的原则。假设每一个闪存块包含  $q$  个页面，且这  $q$  个页面分为  $n$  组，每一组包含  $q/n$  个页面。再假设读取过一段时间之后，每一组累积了  $m$  次读取操作，触发读重写发生的条件是每个闪存块累积  $n * m$  次读取操作。如果一个闪存块中的数据被持续读取并发生了读重写，且其中  $k$  组页面会被重新回写到一个新闪存块中，则当满足以下条件时新的闪存块不会发生读重写：

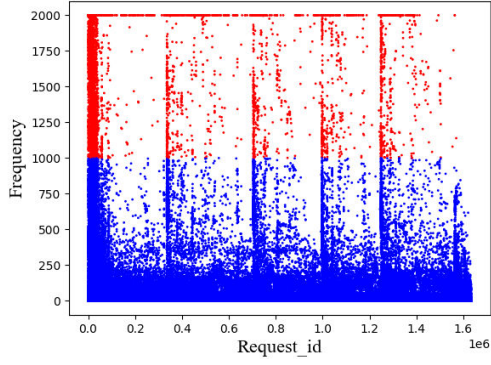
$$k * m + d < n * m, \quad (3.1)$$



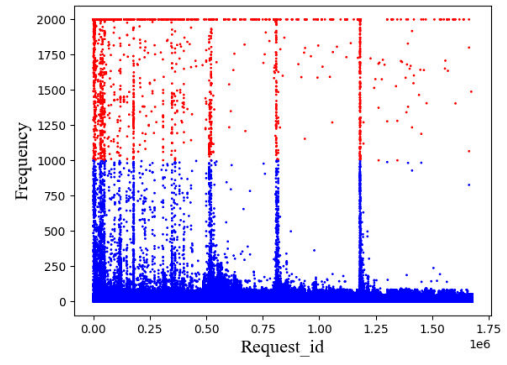
(a) ads1



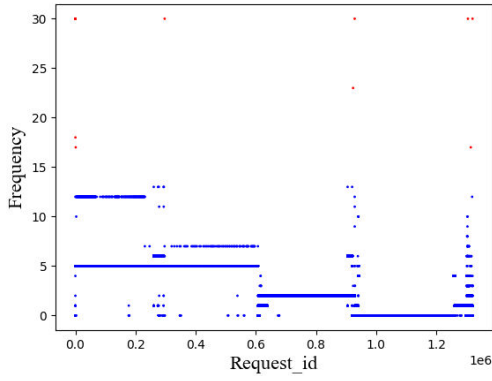
(b) ads2



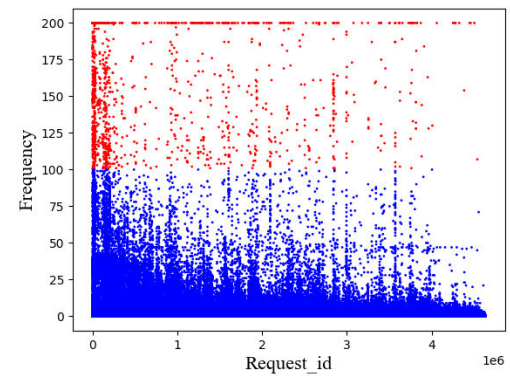
(c) ali206



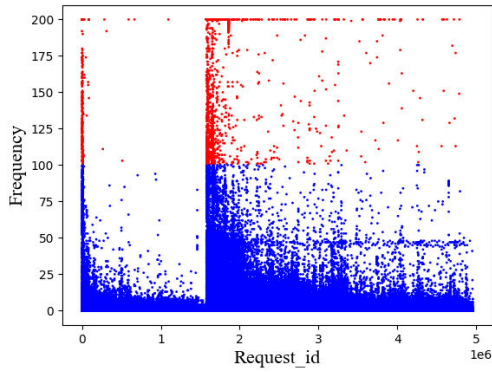
(d) ali188



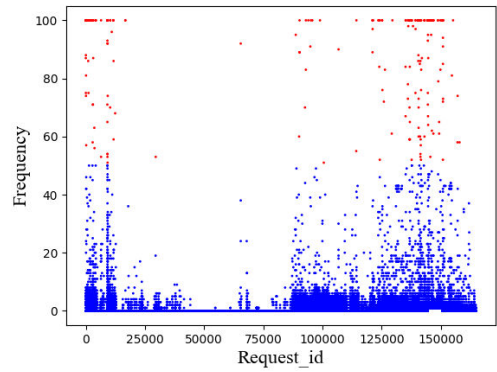
(e) msr\_web



(f) sys1



(g) sys2



(h) nexus5



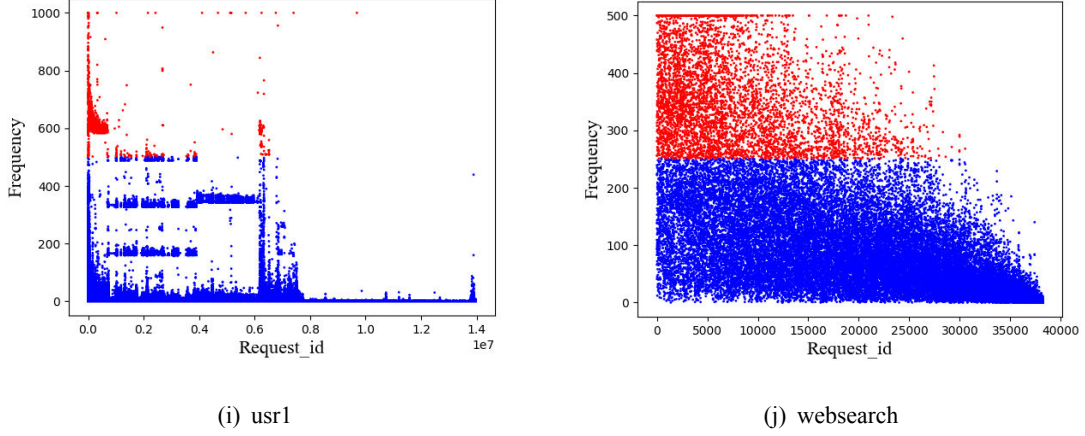


图 3.1 十个不同工作负载的数据访问行为

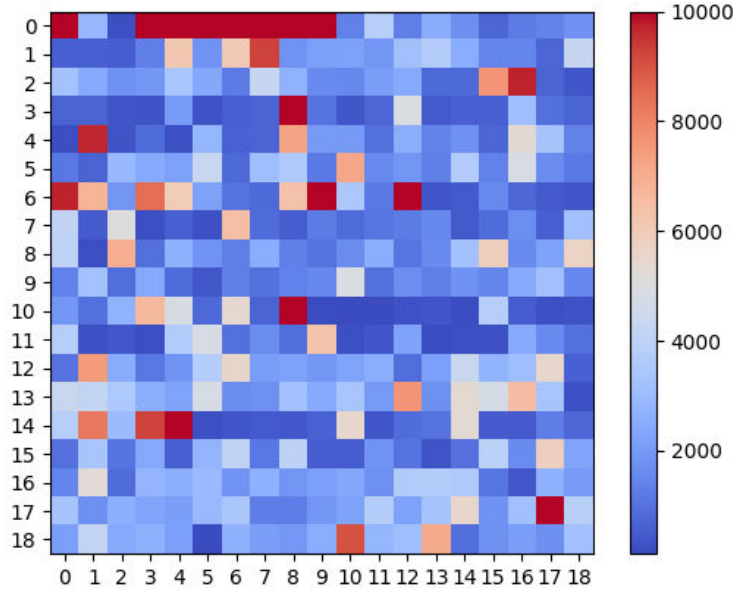


图 3.2 一个 plane 中的数据读取热度分布

其中  $d$  是新闪存块中其他冷数据的读取总数。可以从公式 3.1 中导出三个读重写数据回写时遵循的原则，目的是减少新闪存块的读取总数，从而避免闪存块触发第二次读重写：

- (1)  $d$  的值需要足够小，也就是说，新闪存块中需要有足量的冷读页面；
- (2)  $k$  的值也需要尽可能小，即读重写数据分配到每一个闪存块中的数据量应得当适量；
- (3)  $n$  的值需要尽可能大， $n$  表示新闪存块的数量，即需要尽可能将读重写数据分散开，同时也要求冷读数据的数量应该足够多。

在一个闪存块中放置页面以满足上述三个条件是具有挑战性的。而且，很难预测页面的受欢迎程度（即读热页或读冷页）。Cocktail 的提出就是为了解决这些挑战。它系统地管



理由用户发出的写请求（称为用户写请求）和由读重写触发的写请求（称为读回写请求）引起的写操作。Cocktail 将一个块分成两部分：1) 用户写部分，服务于用户写请求，2) 读回写部分，服务于读回写请求。Cocktail 将读回写请求写入到用户写部分已写完的闪存块，这种块称为 UW-block。如果每个闪存块中读回写部分比上用户写部分的比值较大，那么每个闪存块的读操作就会比较少，从而降低读重写频率。这里有三个主要原因可以解释这种趋势。1) 虽然不同环境下 I/O 轨迹的数据访问行为不同，但仍然可以从大量的用户写数据中提取出一定量的冷读数据<sup>[64-67]</sup>；2) 将读重写回写的页面分散到多个闪存块，不仅减少了闪存块的读取数量，而且增加了其中无效页面的数量；3) 读重写回写页面很可能仍然是热读页面。因此，设计了一个拥有足够数量的 UW-block 的写入池，接着将读回写页面分散在写入池中的多个 UW-block 上，以进一步缓解读干扰问题。

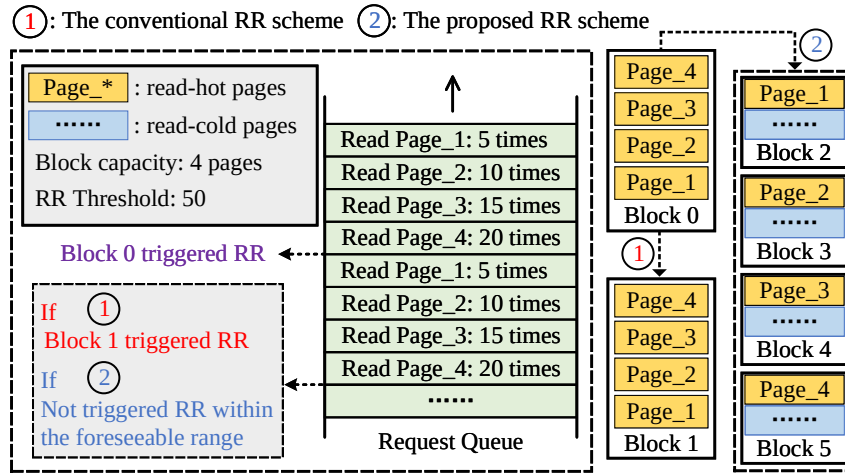


图 3.3 Cocktail 与传统方案数据回写方式的对比

图 3.3 中展示了 Cocktail 和传统读干扰管理方案在读重写数据回写方式上的对比。假设一个闪存块由四个页面组成，并且一个闪存块触发读重写的阈值设置为 50。在图 3.3 所示的请求队列中，当前四个请求得到服务后，Block0 将触发读重写，因为 Block0 的读取总数（即 Page\_1 - Page\_4）等于读重写阈值 50。传统的方案将 Page\_1、Page\_2、Page\_3 和 Page\_4 写入到 Block1 中，如果有一组和之前相同的请求到达，则会导致另一次读重写。然而，Cocktail 将这四个页面写入四个不同的闪存块中，这降低了第二次读重写的可能性。

Cocktail 的首要目标是将读重写产生的热回写数据分散到预先填充了用户写入的冷数据的闪存块上。这里存在两个具有挑战性的问题。首先，准确预测用户写入数据的热度是不可行的；其次，用户写入的每个页面都有自己的访问模式，导致访问行为不同<sup>[64]</sup>。为了应对这两个问题，通过为 Cocktail 配置了双写边界块<sup>[67]</sup>和写入池，可以实现根据读重写的

特点动态调整读重写热数据与用户写入冷数据的比例。特别地, Cocktail 中利用了双写边界块将外部用户写入数据与内部读重写引起的回写数据分开。Cocktail 利用写边界块来调整来自外部用户写请求的相对冷读数据的预填充比例, 将预填充的闪存块保存在相应的写入池中。当读重写热数据到达时, Cocktail 会在写入池中寻找读次数最少的闪存块写入。通过这种方式, 热回写数据可以很容易地被分散开, 并写入冷读数据较多的闪存块中。如此一来, 一个闪存块触发第二次读重写的机率会变得相当低。如章节 3.3.5 所示, 写入池中读取最少的闪存块可能包含有许多无效页面, 每个闪存块中可能会有许多无效页面和少量热读页面放在一起。因此, 即使触发了第二次读重写, 写放大也会大大减少。

### 3.3.2 Cocktail 的设计概述

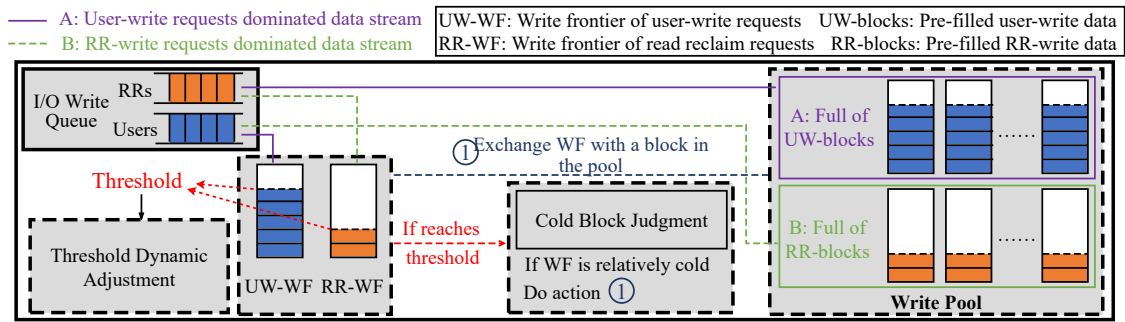


图 3.4 Cocktail 结构总览

Cocktail 的首要目标是通过将读重写引起的热回写数据 (RRs) 分布在预先填充了用户写入数据 (Users) 的闪存块上, 从而减少闪存块的平均读次数。实验部分在测试平台的闪存转换层 FTL 中实现了 Cocktail。Cocktail 会利用双写边界块<sup>[64]</sup>来预填充闪存块, 这有助于创建读平衡块。Cocktail 会维护相应的写入池, 将用户写入数据和读重写数据结合到一起 (如图 3.4 所示)。Cocktail 使用两个动态阈值来控制闪存块中用户写入数据与读重写数据的比例。为了平衡闪存块的读请求访问, Cocktail 会根据两种情况处理回写数据, 分别是用户写密集型情况和读重写密集型情况。

在用户写密集型的情况下, Cocktail 将用户写入数据写到写边界块, 而读重写数据写到写入池中的一个闪存块。Cocktail 会不断向写边界块中写入用户数据, 直到写边界块中的数据量达到设置的阈值。接下来, 如果写入池没有完全填满, Cocktail 将把写边界块放到写入池。否则, Cocktail 会把写边界块与写入池中的每个闪存块进行比较, 以将预期读取次数最少的闪存块放在写入池中。如果写边界块可以放在池中, 那么从池中交换出来的闪存块将接替成为写边界块。在这个过程中, Cocktail 将写入池中的用户数据和读重写数据结合在一起, 形成读平衡块。此外, Cocktail 通过调整两个动态阈值, 动态调整闪存块的

用户数据与读重写数据的比例。另一方面，在读重写密集型的情况下，Cocktail 会执行类似的操作，只是 Cocktail 会将读重写数据写入到写边界块，而同时将用户数据写进写入池的闪存块中。

值得一提的是，Cocktail 会不断更新写入池中的闪存块。这样做有两个目的，如前所述，第一个目的是在写入池中保留相对较冷的读数据，第二个目的则是从写入池中移除读取量较大的闪存块，以防止这些闪存块触发读重写操作。如章节 2 中所述，读重写操作和垃圾回收操作的本质都是擦除闪存块。由于 NAND 闪存中的闪存块在擦除数量上是有限制的，因此在擦除之前应该充分利用每个闪存块。写入池的闪存块由于仍然存在可用空间，如果让这些闪存块触发读重写，则会浪费其中的一部分空闲空间。因此，Cocktail 会及时从写入池中移除读取量较大的闪存块，并接收用户写入的数据，从而确保这些闪存块在被擦除之前得到充分利用。

### 3.3.3 双写边界块

双写边界块策略被广泛应用于通过分离内部和外部写请求来减少写放大。Cocktail 可以利用双写边界块来预填充数据，以便在写入池中对数据进行筛选。根据回写数据的特性，Cocktail 用给定的比例给写边界块预先填充用户数据或者读重写数据。读重写边界块（RR-WF）用于接收读重写数据，用户写边界块（UW-WF）用于接收用户写入数据。由于现代 NAND 闪存支持平面级（plane）并行，因此 Cocktail 为每个 plane 设置两个写边界块，每个 plane 都与一个独立的写入池相关联。Cocktail 还同时维持一个空闲闪存块链表，如果写边界块被预先填充到设定阈值，Cocktail 将尝试用写入池中的一个闪存块来跟当前写边界块进行交换。或者如果写边界块已经被填满，那么 Cocktail 将从空闲闪存块链表中选择一个闪存块，使其成为新的写边界块。

为了控制预填充闪存块中用户数据和读重写数据的比例，Cocktail 中引入了两个阈值  $TH_{user}$  和  $TH_{rr}$ 。由于 Cocktail 中的读平衡块仅由用户写入数据和读重写数据组成，因此 Cocktail 中用  $TH_{user} = (1 - TH_{rr})$  来保证闪存块中读重写数据与用户数据的比例保持不变，无论是用户写密集型情况还是读重写密集型情况。如前所述，在用户写密集型的情况下，Cocktail 会将用户数据预先填充写边界块，如果块中的数据量达到  $TH_{user}$ ，Cocktail 将停止预填充写边界块，并将其放进写入池等待接收读重写数据，生成读平衡块。Cocktail 在读重写密集型的情况下，也会对读重写数据的写边界块进行类似的操作。

### 3.3.4 写入池

Cocktail 中建立了一个写入池，其中用户写入数据和读重写数据被写入预填充的闪存块，形成读平衡块。并且，Cocktail 还为写入池中的每个闪存块设置一个标志，用来指示一个闪存块是预先填充了用户数据还是读重写数据。如果一个闪存块是预先用用户数据填充的，那么 Cocktail 将在写入池中用读重写数据填充该块，反之亦然。

Cocktail 中使用了滑动窗口协议来确定写入池容量。滑动窗口有一个最小值和一个最大值，Cocktail 根据实际实验情况设定了一个最小值。具体来说，最小值被确定为 plane 中闪存块数量的 1%。例如，一个 plane 如果有 2048 个闪存块，则写入池的容量为 20 个闪存块。如前面 3.2 章节中所述，同一 plane 中只有少数闪存块会受到严重的读干扰，而大多数闪存块几乎不受读干扰的影响。此外，实验部分计算的读重写触发时每个闪存块的读取数量（热分布图如图 3.2 所示）也表明，较小的比例可以更好地保留冷数据。

当写入池已被填满，且用户写边界块达到阈值时，Cocktail 将写入池的容量增加一，并将写边界块放入到写入池中。当写入池应对了一次读重写数据的回写操作，且写入池的剩余闪存块数量小于最小值时，写入池容量会重新设置为最小值，以便可以被快速填满。关于写入池容量最大值的确定，回顾章节 3.3.2 中的内容，写入池中的闪存块应该避免触发读重写操作。如果写入池容量过大，一些读次数较多的闪存块可能不可避免地在写入池中触发读重写。而且由于确定最大值与确定不同生产环境中统一的超额配置一样困难<sup>[68-69]</sup>，Cocktail 将最大值设置为 plane 中闪存块数量的 3%。第五节的实验结果表明，这样的配置使得 Cocktail 在十种不同的轨迹下有良好表现。

接下来将从处理用户数据和处理读重写数据这两个角度演示 Cocktail 如何生成读平衡块。

处理用户数据的主要目标是把这些数据与预填充的读重写数据组合成读取平衡的闪存块。为此，Cocktail 中设计了帮助处理用户数据的算法 1。如果找到这样的块，Cocktail 将用户的写入数据填充到找到的闪存块中，并将新写入的用户数据与预填充的读重写数据结合，生成一个读平衡块（参见算法 1 第 37 行）。否则，如果写入池中没有这样的闪存块，Cocktail 会将用户数据写入到写边界块中，直到达到预定义的阈值 THuser（参见算法 1 中的第 13 - 14 行）。接下来，当达到 THuser 时，Cocktail 会检查写入池，确定池中的闪存块数量是否已达到其容量。如果写入池未满，Cocktail 会将这个写边界块直接放入池中，然后从空闲块链表中申请一个空闲块，使其成为一个新的写边界块。如果池已满，Cocktail 会将写边界块和写入池中的闪存块进行比较，留下数据较冷的闪存块。因此，未来读次数较

少的闪存块将留在写入池中，而未来读次数最多的闪存块将作为写边界块（参见算法 1 的第 15 - 35 行）。值得注意的是，有两个因素会显著影响一个闪存块的未来读次数。一个是当前闪存块的读取数量，另一个是闪存块中无效页面的数量。

---

**Algorithm 1** 用户数据写入策略
 

---

```

1: /*RR_blocks are pre-filled RR-write data*/
2: /*WR_blocks are pre-filled user-write data*/
3: if RR_block can be found in pool then
4:     target_block = RR_block
5: else
6:     target_block = nullptr
7: end if
8: /*wf is the write frontier of user-write data*/
9: /*bkpool is the blocks number of pool*/
10: /*numblocks is the blocks number of plane*/
11: /*numpages is the pages number of block*/
12: if target_block == nullptr then
13:     write user-write data to wf
14:     wf.index++
15:     if bkpool < max_numblocks then
16:         if wf.index ≥ THuser * numpages then
17:             /*It will be a WR_block*/
18:             wf.status = StateWR
19:             pool.push(wf)
20:             wf = get_a_new_block()
21:             check GC required
22:         end if
23:     else
24:         if wf.index == numpages then
25:             wf = get_a_new_block()
26:             check GC required
27:         else if wf.index ≥ THuser * numpages then

```

---

```

28:         Find(target_block)
29:         if target_block != nullptr then
30:             pool.remove(target_block)
31:             pool.push(wf)
32:             wf = target_block
33:         end if
34:     end if
35: end if
36: else
37:     write user-write data to target_block
38:     target_block.index++
39: end if

```

---

Cocktail 在处理读重写数据时遵循与处理用户数据时类似的过程来生成读平衡块。算法 2 中给出了 Cocktail 处理读重写数据过程的伪代码。具体来说，当读重写数据到达时，Cocktail 将首先在写入池中搜索一个预先填充了用户数据的闪存块（参见算法 2 中的第 3 行）。如果找到这样一个闪存块，Cocktail 将到达的数据写入找到的块中；如果没有找到，Cocktail 将到达的数据写入到读重写边界块中。接下来，当写边界块达到读重写阈值  $TH_{rr}$  时，如果写入池未滿，Cocktail 将首先尝试将其放进写入池中（参见算法 2 中的第 35 行，13 - 25 行）。如果写入池中都是预先填充了读重写数据的闪存块，则意味着读重写数据占据数据流的主导地位，导致大量读重写数据写入。为了避免污染太多的闪存块，Cocktail 会将数据聚合到一部分闪存块中。具体来说，Cocktail 会从写入池中选择读取次数最少的闪存块，然后将到达的读重写数据写入其中（参见算法 2 中的第 27 - 32 行）。

---

#### Algorithm 2 读重写数据写入策略

---

```

1: /*RR_blocks are pre-filled RR-write data*/
2: /*WR_blocks are pre-filled user-write data*/
3: if WR_block can be found in pool then
4:     target_block = WR_block
5: else
6:     target_block = nullptr
7: end if
8: /*wf is the write frontier of RR-write data*/

```

```
9: /* $bk_{pool}$  is the blocks number of pool*/
10: /* $num_{blocks}$  is the blocks number of plane*/
11: /* $num_{pages}$  is the pages number of block*/
12: if  $target\_block == nullptr$  then
13:     if  $bk_{pool} < max\_num_{blocks}$  then
14:         write RR-write data to  $wf$ 
15:          $wf.index++$ 
16:         if  $wf.index == num_{pages}$  then
17:              $wf = get\_a\_new\_block()$ 
18:             check GC required
19:         else if  $wf.index \geq TH_{rr} * num_{pages}$  then
20:             /*It will be a  $RR\_block$ */
21:              $wf.status = state_{RR}$ 
22:              $pool.push(wf)$ 
23:              $wf = get\_a\_new\_block()$ 
24:             check GC required
25:         end if
26:     else
27:         Find( $target\_block$ )
28:         write RR-write data to  $target\_block$ 
29:          $target\_block.index++$ 
30:         if  $target\_block.index == num_{pages}$  then
31:              $pool.remove(target\_block)$ 
32:         end if
33:     end if
34: else
35:     write RR-write data to  $target\_block$ 
36:      $target\_block.index++$ 
37: end if
```

---

### 3.3.5 读重写预测因子

写入池中闪存块的替换原理是将未来读次数可能较少的闪存块保留在池中。尽管要知道一个闪存块的未来读取数量是不可行的，但通过实验可以观察到有两个关键因素对读重写有重大影响。这两个关键因素就是闪存块的当前读取总数和闪存块中当前的有效页面数量。为了便于预测读重写的触发数量，Cocktail 引入了一个读重写预测因子，它表示了上述两个关键因素的共同影响。

$$factor = ratio * (RC/TH) + (1 - ratio) * (Pv/Pa), \quad (3.2)$$

其中 TH 为足够触发读重写的读次数阈值；RC 表示当前闪存块的读取次数；Pv 和 Pa 分别表示当前闪存块有效页面的数量和闪存块中所有页面的数量。通过计算  $RC/TH$  和  $Pv/Pa$ ，可以将读取次数和有效页面这两个不同的参数归一化。ratio 是一个附加变量，通过设定 ratio，可以调整每个闪存块的预测因子。这个预测因子决定了 Cocktail 写入池替换策略的替换标准。

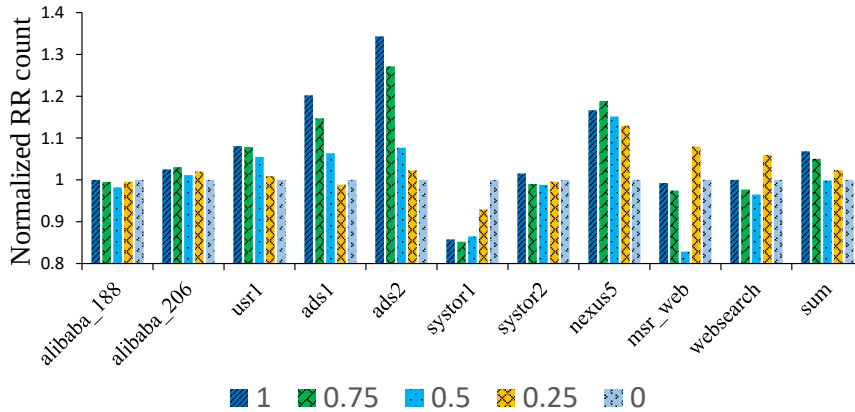


图 3.5 不同预测因子触发的读重写总数

为了找出减少读重写数量的最佳预测因子，实验部分在十个真实工作负载下对不同预测因子进行了对比。通过以 0.25 为增量将预测因子中的 ratio 定量的从 0 变为 1 来研究预测因子对读重写的影响。由图 3.5 可知，在五种情况中（即 1, 0.75, 0.5, 0.25, 0），0.5 是预测因子的最佳配置。这个发现是可预料的。图 3.5 中显示，在大多数工作负载条件下，0.5 情况下导致的读重写次数最少。此外，由于闪存块读取的数量和无效页面的数量都对减少读重写次数有重大影响，因此将这两个因素同等考虑是明智的。如前所述，不同的工作负载具有不同的数据分布，因此在实际生产环境中，该参数还可以根据数据特征进行调整。在本文中，Cocktail 将这个闪存块的读取次数与有效页面数量的比率统一设置为 0.5。



### 3.3.6 读重写数据和用户写入数据的比例关系

用户写入数据和读重写数据在一个闪存块中适当的比例可以创建读取均衡的闪存块,从而降低读重写的触发数量。为了深入了解数据比例的影响,实验部分还进行了一组对比实验来获得用于生成读取平衡闪存块的最佳数据比例。如 3.3.1 章节所述,假设一个闪存块被分成  $n$  部分,每个闪存块由  $1/n$  的读重写数据和  $(n-1)/n$  的用户写入数据组成。图 3.6 中的实验结果是在十种工作负载条件下测试了不同  $n$  值的读重写次数。

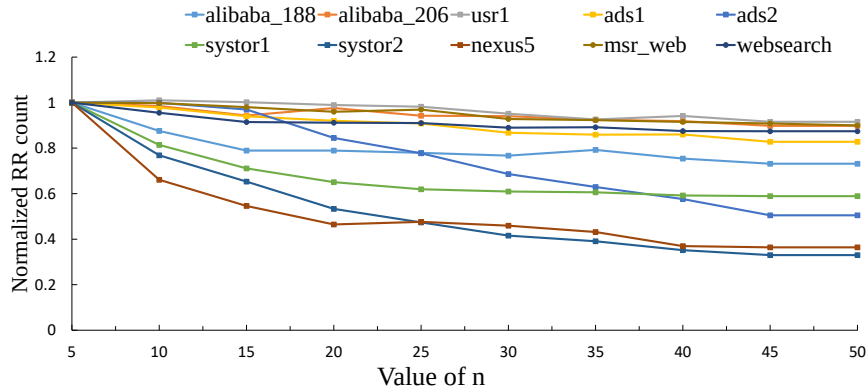


图 3.6 不同  $n$  值触发的读重写次数

从图 3.6 可以看出,随着读重写数据比例的降低,读重写次数也随之下降,最终呈现平缓趋势。在选择  $n$  来达到最佳配置时,除了图 3.6 的结果外,还有其他因素值得考虑。特别是,如果写入池中只有很少的闪存块,可以推导出当前数据流只有很少的用户写入数据。较小的  $n$  值可以帮助 Cocktail 提前将写边界块放进写入池中,与读重写数据结合,创建读取均衡的闪存块。相反,如果写入池已满,可以推断当前数据流中用户写入的数据远远多于读重写数据。因此,可以使用一个比较高的  $n$  值来选择“更高质量”的闪存块来组合读重写数据。

在 Cocktail 中,不会使用固定的  $n$  值,而是动态地调整闪存块中用户数据和读重写数据的比例。假设写入池最多可以存储  $m$  个闪存块,当有读重写发生且被擦除的闪存块中的页面都有效时,为了保证所有读重写数据在写入池中都有可写的位置,写入池中的每个闪存块都需要预留  $1/m$  的空间,即  $n = m$ 。如 3.3.4 章节所述,写入池的容量是动态变化的。当  $n = m$  时, $n$  的值可以随着写入池容量的变化而动态变化。例如,如果写池中有 20 个块,则每个块中用户数据的比例为  $19/20$ ,而读重写数据的比例为  $1/20$ 。即两种数据的比例是写入池容量的倒数。3.4 章节的最终实验结果表明,Cocktail 在十种不同的工作负载下都有良好的表现。

### 3.3.7 性能分析

在多个工作负载 (阿里巴巴在 2020 年公布的 1000 个虚拟机中的读写请求情况<sup>[70]</sup>) 中可以观察到, 真实世界的工作负载都是即包含读请求也包含写请求的。在许多其他场景<sup>[71-74]</sup>中的工作负载也包含了很大比例的写请求。在这些场景中, Cocktail 可以从写入数据中充分地保存相对较冷的数据。这些保存下来的冷读数据可以与读重写操作产生的热读数据相结合。虽然 Cocktail 无法在只有读请求的工作负载中有特别突出的表现, 但是这可以通过在 FTL 中的选择机制选择其他数据管理算法进行解决。简而言之, Cocktail 主要关注的是读请求和写请求共存的场景。

接下来将从计算开销和内存需求两方面分析 Cocktail 的算法开销。Cocktail 的主要计算开销是当用户数据到达时, 在写入池中找到最合适的闪存块。由于写入池的大小被设置得相对较小 (例如一个 plane 中闪存块数的 2%), 这样的搜索可以非常快被完成。值得注意的是, 无论闪存转换层如何设计, 都需要对每个读请求执行闪存块读计数器的更新和检查。上述操作一般以纳秒为单位, 而 NAND 闪存的相关操作一般是以微秒为单位<sup>[38,75-76]</sup>。因此, Cocktail 中所提出的设计方案只会导致可以忽略不计的计算开销, 那些与 NAND 闪存相关的基本操作 (即由于读干扰而移动不可靠的页面) 需要更长的时间。另外, Cocktail 不会占用额外的存储空间。Cocktail 中的写入池是指针数组的一个简单实现, 因此, 当闪存存储空间不足以存储数据时, Cocktail 甚至可以将指针数组的长度减少到 0。对于内存需求, Cocktail 中的写入池只需要通过指针指向实际的块, 这意味着写入池的设计不会占用太多额外的内存开销。

## 3.4 实验验证及分析

### 3.4.1 实验设置

为了评估 Cocktail 的性能, 本文在模拟器 MQSim<sup>[77]</sup>中实现了 Cocktail。MQSim 是一个开源的 NAND 闪存固态模拟器, 能够精确模拟现代 NAND 闪存的评估指标。研究 Cocktail 需要在源码的基础上添加读重写机制, 设置读重写可以根据闪存块的读次数进行触发, 并且将每个闪存块触发读重写的阈值设置为页面数量的 40 倍<sup>[11]</sup>。以一个闪存块 256 个页面为例, 当闪存块总的读取次数达到 10240 次之后, 闪存系统就会对其执行擦除操作。表 3.1 总结了 MQSim 模拟器的配置细节。

本文在十个不同读写比例的真实工作负载上进行实验 (见表 3.2), 其中 alibaba\_188 和 alibaba\_206 是阿里巴巴某个数据中心两台虚拟机在 2020 年其中一个月的访问行为<sup>[70]</sup>。

表 3.1 NAND 闪存模拟器实验配置

|                         |                         |
|-------------------------|-------------------------|
|                         | 8 channels              |
| SSD organization        | 4 chips per channel     |
|                         | 2 dies per chip         |
|                         | 2 planes per die        |
| Flash Microarchitecture | 2048 blocks per plane   |
|                         | 256 pages per block     |
|                         | Page size: 8 KB         |
|                         | Read latency: 75 us     |
| Flash Latencies         | ECC latency: 20 us      |
|                         | Program latency: 750 us |
|                         | Erase latency: 3.8 ms   |
| GC Parameters           | Policy: GREEDY          |
|                         | Threshold: 0.28         |

systor1 和 systor2 分别记录了企业虚拟桌面基础设施在 2017 年中两个不同月份的数据访问行为<sup>[71]</sup>。nexus5 是智能手机的数据访问行为<sup>[72]</sup>。ads1 和 ads2 来自微软的某个广告平台<sup>[73]</sup>。usr1 和 msr\_web 来自企业集群<sup>[74]</sup>。websearch 来自 UMass 的存储库。

由于读重写操作仅在读密集型任务中触发，因此在小型设备或较短的收集时间中收集的工作负载可能不会触发足量的读重写操作。为了让实验对比结果更加明显，本文将这些工作负载的执行次数增加了 N 倍，其中 N 是表 3.2 中所示的 Repeat Times。本文还根据图 3.1 所示的十个工作负载的数据访问行为，增加了每个工作负载的读请求密度，利用读请求的重复次数（RAF）来确保每个工作负载都有相似的读请求频率。

表 3.2 工作负载读写比例和放大情况

| Trace              | Read Ratio | RAF | Repeat Times |
|--------------------|------------|-----|--------------|
| <i>alibaba_206</i> | 0.8648     | 2   | 1            |
| <i>alibaba_188</i> | 0.6539     | 4   | 1            |
| <i>usr1</i>        | 0.9146     | 8   | 1            |
| <i>systor1</i>     | 0.6217     | 30  | 5            |
| <i>systor2</i>     | 0.5996     | 30  | 5            |
| <i>ads1</i>        | 0.9052     | 40  | 10           |
| <i>ads2</i>        | 0.5625     | 40  | 10           |
| <i>nexus5</i>      | 0.3376     | 50  | 10           |
| <i>msr_web</i>     | 0.9925     | 50  | 10           |
| <i>websearch</i>   | 0.9997     | 50  | 10           |

### 3.4.2 对比实验介绍

本文通过将 Cocktail 与三种不同的解决方案进行比较来评估其性能。将双写边界块方法设置为基线解决方案,此外,将 Cocktail 与另外两种读干扰管理方案 redFTL<sup>[18]</sup>和 IPR<sup>[23]</sup>进行了对比评估。

双写边界块方法利用不同写边界块将用户数据和读重写数据写入不同的块。redFTL 通过向每个页面的逻辑地址中添加一个字节来记录页面的读取次数。基于这些额外的信息,redFTL 将读重写数据结合起来并写入同一个读次数较少的闪存块中,在执行读重写时创建读干扰闪存块。redFTL 通过改变读干扰闪存块的读电压,将读干扰闪存块的读重写阈值提高到正常读重写阈值的两倍,其目的是避免读干扰闪存块的第二次读重写发生。当读干扰闪存块中再次触发读重写时,读重写数据将被写入读重写阈值为正常读重写阈值 5 倍的闪存块中。IPR 则是通过 monitor-block 和 IPR-block 来创建读干扰闪存块。monitor-block 监视每个页面的读计数。在进行读重写时,如果某个闪存页面的读次数大于平均读次数,则该闪存页面将被重写到 IPR-block。否则,该页面将被写回普通闪存块。与 redFTL 类似,IPR 也通过改变这些闪存块的读电压,将 IPR-block 的读重写阈值提高到正常阈值的 10 倍。monitor-block 占总容量的 10%,IPR-block 占总容量的 15%。

Cocktail 与 redFTL 和 IPR 明显不同,Cocktail 保留每个闪存块的一小部分,以便将来与热读重写数据组合,创建读平衡的闪存块,从而将热读重写数据分布在广泛的闪存块上。与 redFTL 和 IPR 将所有热数据集中到一小部分闪存块中不同,Cocktail 更有利于减少读重写次数,而且还减少了 NAND 闪存的响应时间和擦除操作的数量。众所周知,读请求所需的时间主要取决于存储介质的读取速度。减少闪存块擦除的数量可以最小化写放大对用户请求的影响。此外,缓解读干扰也可以减少 ECC 校验机制的重试次数,从而进一步提高 I/O 效率。接下来将分成三个部分描述实验结果:1) 读重写次数,2) NAND 闪存响应时间,3) 写放大和垃圾回收情况。

### 3.4.3 读重写情况分析

在这一章节中,主要研究 Cocktail 对 NAND 闪存使用寿命的影响,并且将 Cocktail 与基线管理方案(即双写边界块)、redFTL 和 IPR 在十种工作负载条件下的读重写次数进行了比较。由于不同工作负载的实验结果的尺度不尽相同,通过将基线管理方案的结果设置为 1,可以将读重写计数结果归一化,并按比例缩放 redFTL、IPR 和 Cocktail 的实验结果。

图 3.7 给出了四种算法在十种不同工作负载条件下的归一化读重写计数。与其他三种算法相比,Cocktail 在多个工作负载条件下有更好的性能。特别是在工作负载 usr1 的情况

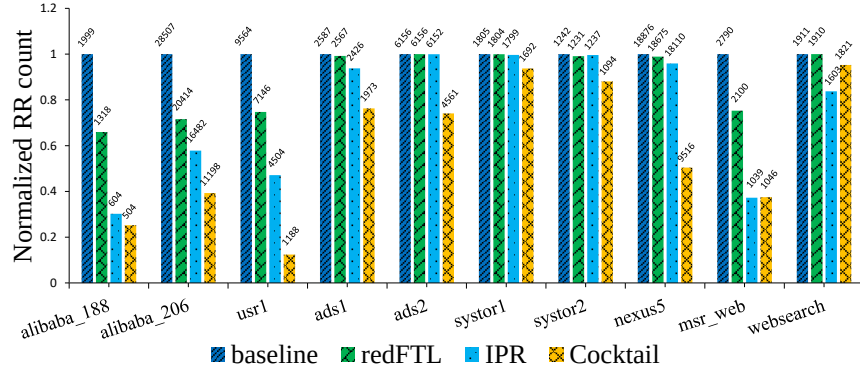


图 3.7 平均读重写计数

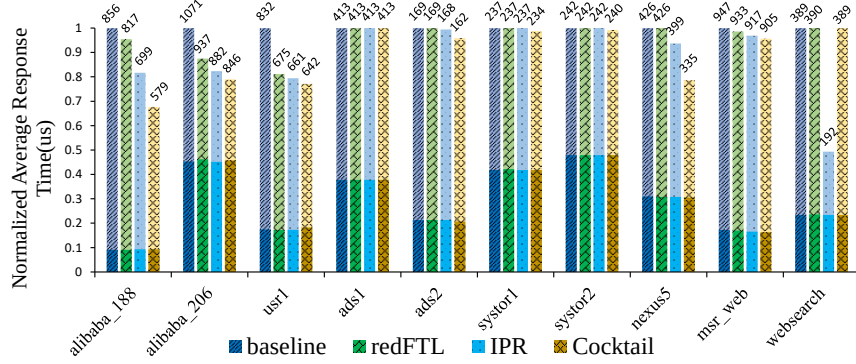
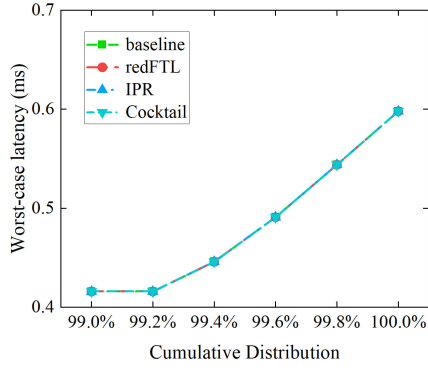


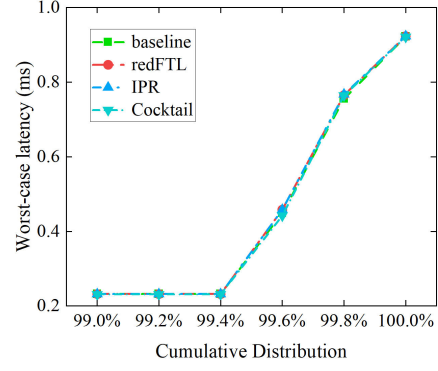
图 3.8 平均反应时间

下, Cocktail 的读重写计数比其他三种方案平均减少了 80% 以上。Cocktail 这样的性能表现是可以预见的。redFTL 和 IPR 中将读重写数据集中放置的这一策略容易导致数据在聚集后再次引起严重读干扰,从而触发第二次读重写操作。与这些解决方案不同, Cocktail 通过将读重写数据与用户写入的较冷数据相结合,明智地将读重写数据分布在广泛的闪存块上,从而显著地避免了第二次读重写的发生。此外, Cocktail 中的去中心化放置策略也减少了整体的闪存块读取。但在只有读请求的场景下, Cocktail 可能没有足够的用户数据与读重写数据进行组合。虽然 Cocktail 中有处理这种情况的机制 (3.3.4 章节中的 2),但是效果没有特别的突出。在实验结果中也可以发现, Cocktail 在 msr\_web 和 websearch 两个基本只读工作负载中,性能略差于 IPR,但还是优于基线方案和 redFTL。

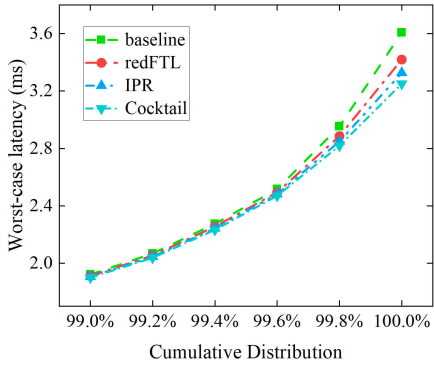
简而言之, Cocktail 不仅减少了由集中放置引起的第二次读重写的几率,而且还减少了整体闪存块的读取次数。这些特征使得 Cocktail 能够在读重写次数上比 baseline、redFTL 和 IPR 平均降低了 40.77%、36.06% 和 21.75%。



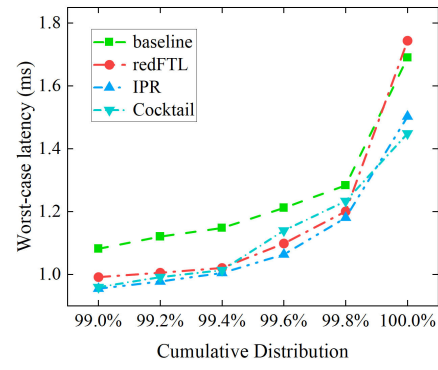
(a) ads1



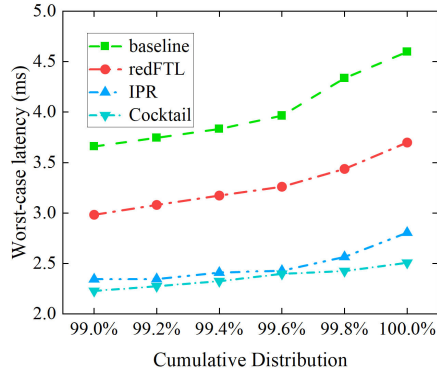
(b) ads2



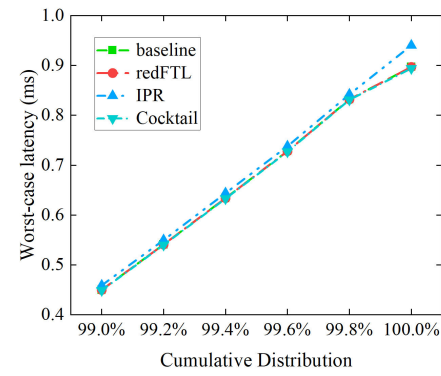
(c) ali206



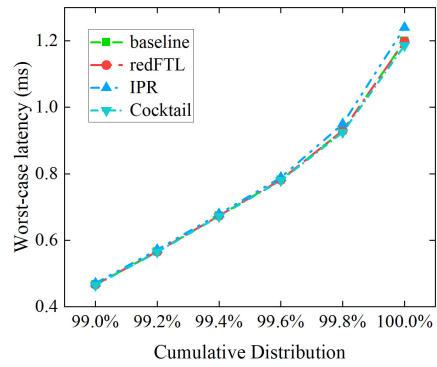
(d) ali188



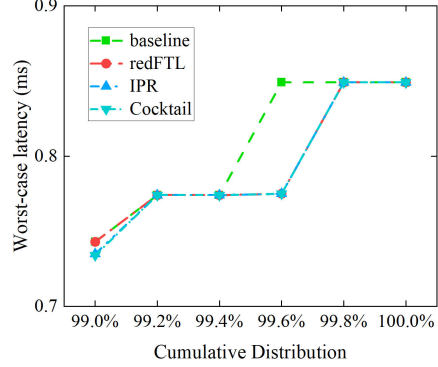
(e) msr\_web



(f) sys1



(g) sys2



(h) nexus5

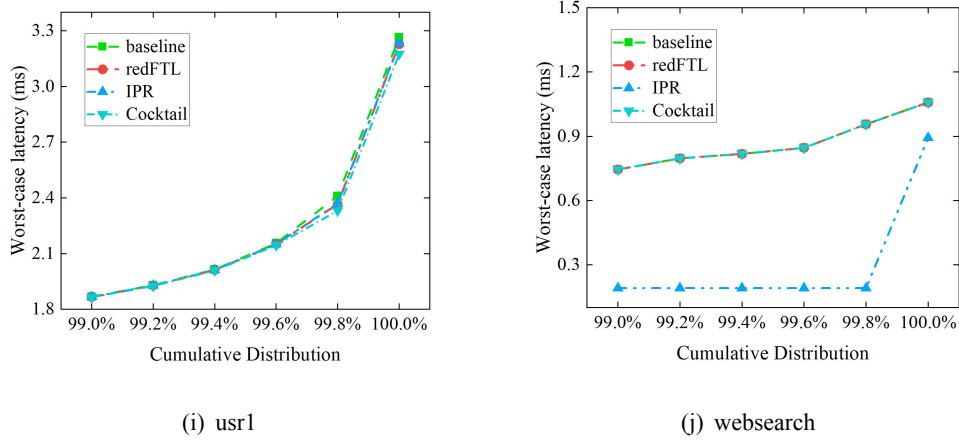


图 3.9 不同工作负载的尾延迟情况

### 3.4.4 反应时间情况分析

本文注重 Cocktail 在 NAND 闪存响应时间上的表现。因此，实验中在十种工作负载条件下观察了 Cocktail、baseline、redFTL 和 IPR 的平均反应时间和尾延迟情况。读干扰经常会导致读操作需要多个重试步骤。在 0 擦除周期（即 NAND 闪存生命周期开始时）的 3 个月数据保留周期下，每次读取都需要超过 3 个重试步骤<sup>[7]</sup>。基于这一发现，本文在 MQSim 模拟器做了如下改动：当一个闪存块的读取计数达到读重写阈值的 70% 时，重试 ECC 校验一次，当达到 80% 时重试两次，当达到 90% 时重试三次。实验结果如图 3.8 所示。直方图中较暗的部分表示每个请求的平均 ECC 校验重试时间。它表明，在 ali206、systor1 和 systor2 这三种情况下，平均 ECC 校验重试时间占平均响应时间的近一半。

图 3.8 和图 3.9 显示了四种数据管理方案在十种工作负载上的平均响应时间和 99% 到 100% 的尾部延迟。结果表明，在 ali206、ali188、msr\_web 和 nexus5 这些工作负载条件下，Cocktail 在平均响应时间和尾延迟方面的表现比 baseline、redFTL 和 IPR 都要突出。而在 ads1、ads2、systor1 和 systor2 这几个工作负载的测试中，Cocktail 与其他三种方案相比没有明显优势。这样的实验结果是合理的。这是因为 NAND 闪存在读重写过程中必须将数据从当前闪存块移动到其他闪存块。NAND 闪存会努力确保操作的原子性，这不可避免地导致了后续读请求的延迟。在读密集型条件下工作时，这样的延迟会变得更加严重。与 Cocktail 相比，传统的解决方案在除 websearch 以外的工作负载中减少的读重写计数较少。在 websearch 负载中，IPR 的表现会略好于 Cocktail，而在其它负载中则是 Cocktail 的性能比较突出，这也导致了他们的平均响应时间比起 Cocktail 要更长一些。



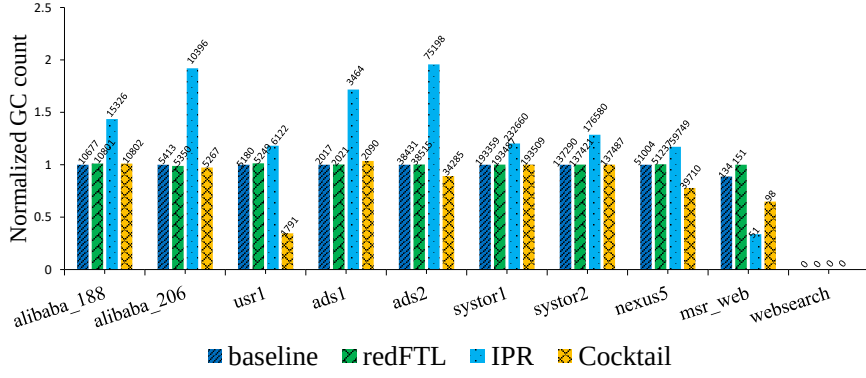


图 3.10 平均垃圾回收次数

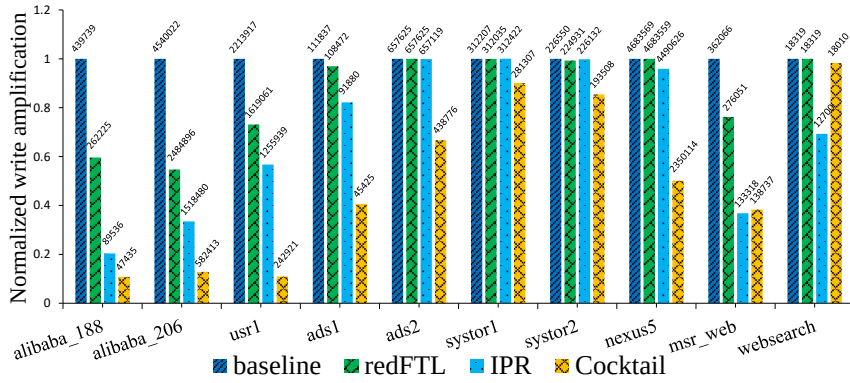


图 3.11 平均写放大

### 3.4.5 写放大和垃圾回收情况分析

这一章节将从写放大和垃圾回收次数这两个角度来评估 Cocktail 的性能。同样的，会在十个工作负载中，将 Cocktail 与其他三种解决方案进行比较。从图 3.10 和图 3.11 的实验结果可以看出，虽然 Cocktail 主要关注的是读干扰管理而不是垃圾回收优化，但是 Cocktail 同样减少了擦除操作造成的写放大和垃圾回收的数量。

垃圾回收算法的一个重要目标是减少写放大。出于这个原因，现有的研究已经提出了许多垃圾回收算法来达成这个目标。例如，FIFO GC 算法 [17] 使用了循环选择闪存块进行擦除的原则，最容易实现但是减少写放大的效果最差。Greedy GC 算法<sup>[69]</sup>是为了在均匀随机写场景下最小化写放大而设计的，它选择包含最少有效页数的闪存块进行擦除。Windowed GC 算法<sup>[68]</sup>是基于 FIFO GC 算法的简单性和 Greedy GC 算法的优越性能而设计的。Van 等人则开发了 d-choices GC 算法<sup>[78]</sup>，旨在从一组随机选择的 d 个块中选择有效页数最少的闪存块进行擦除。在这其中 Greedy GC 算法和 d-choices GC 算法的性能最好，由于即使对于中等的 d 值（如  $d = 10$ ），d-choices GC 算法的性能也略差于 Greedy GC 算法，因此 Cocktail 中采用了 Greedy GC 算法来有效减小写放大。



图 3.10 描述了四种方案的垃圾回收计数。从图中可以看出，在不同的工作负载下，Cocktail 的垃圾回收触发数量比其他三种方案都要少。具体来说，Cocktail 触发的垃圾回收数量比 baseline、redFTL 和 IPR 平均分别减少 12.29%、16.21% 和 22.40%。在工作负载中测试时，IPR 在 `usr1` 中的性能最差、拥有最高的垃圾回收计数。主要原因是 IPR 虽然提高了读重写阈值，但是它牺牲掉了一部分闪存块空间，且这部分闪存块只用于存储热读数据。因此，在写入相同数量的数据时，IPR 由于其较小的空间而触发了更多的垃圾回收数量。

图 3.11 展示了四种对比方案的写放大情况。结果表明，Cocktail 在写放大上优于其他三种方法：Cocktail 算法在擦除过程中移动的平均有效页面数小于其他方案。具体来说，与 baseline、redFTL 和 IPR 相比，Cocktail 的写放大分别降低了 49.57%、45.99% 和 29.93%。简而言之，Cocktail 通过大幅削减读重写的数量而有效地降低了写放大。特别是导致读重写的闪存块中的大多数数据都是热读数据。这些热读数据可能会在闪存块中持续保持有效状态，直到闪存块被擦除。读重写引起的写放大比垃圾回收引起的写放大更严重。因此，Cocktail 减少了读重写的数量就可以大大减轻 NAND 闪存的写放大。

### 3.5 本章小结

这一章节提出了一种高效、低成本的读干扰管理技术 Cocktail。Cocktail 可以减少读重写的数量，从而减轻读干扰的影响。Cocktail 提取用户写入的冷数据，并将其与读重写引起的回写数据相结合，通过将热点数据分布在大范围的闪存块中，避免了热点数据集中在少数闪存块中。最后在闪存模拟器 MQSim 上对 Cocktail 进行了实验，并将 Cocktail 与现有的三种方案 double write frontier、redFTL 和 IPR 进行对比，实验结果表明，与现有方案相比，Cocktail 的读取回收次数、平均响应时间、99% 尾延迟、GC 次数和写放大平均减少了 40.77%、10.82%、5.40%、12.29% 和 49.57%。

## 4 基于读干扰感知的数据缓存策略 Minato

在读密集型工作负载环境下，读干扰会严重降低 SSD 的性能，甚至缩短 SSD 的使用寿命。通常，闪存转换层（FTL）可以维持一个读写缓冲区，在有限的 DRAM 空间中存储频繁访问的数据。本章节提出了一种可感知读干扰的数据缓存管理方案 Minato，该方案根据闪存块的读干扰级别来缓存读请求，从而降低整体读干扰。由于闪存的异地更新操作导致的垃圾回收和读写请求之间执行时间的不一致，写请求对 NAND 闪存的影响比读请求更大。Minato 的读写缓存空间分配方法会优先考虑不影响写请求的命中率。实验部分在九个实际工作负载条件下测量了 Minato 的写请求缓存命中率和整体读性能。实验结果表明，与另外两种缓存策略相比，Minato 可以在不影响写请求命中率的情况下缓解闪存的整体读干扰、减少读重写次数和读反应时间。

### 4.1 引言

3D NAND 闪存凭借体积小、功耗低、访问速度快等优点，目前已被广泛应用于存储设备。得益于 TLC、QLC 等制作工艺的发展，相同体积的 NAND 闪存设备可以存放的数据越来越多。然而，先进工艺在带来更大容量的同时，随之而来的是难以忽视的可靠性问题。读干扰就是 NAND 闪存系统中影响可靠性的一个主要因素。读干扰的主要成因是读积累。当读取闪存块中的一个页面时，NAND 闪存必须对同一闪存块中的所有其他页面施加一个直通电压。这个直通电压的电压值比读参考电压大很多，略低于编程电压。因此，重复施加直通电压就像不断进行弱编程操作，这种操作在读取次数过多后会严重扰乱闪存块的电压分布。当闪存页面读取的比特错误大于 ECC 纠正能力时，NAND 闪存就必须改变读参考电压并重复读取操作，直到页面的数据被正确读取（详见章节 2）。因此，当闪存块受到严重读干扰时，读取该闪存块上的页面所需的时间将延长数倍。

当前已经提出了很多缓解读干扰的方法。读重试有一个原因是因为 ECC 机制的校验能力不足，因此可以通过使用更强有力的 ECC 校验模式<sup>[48-51]</sup>，容纳更多的读错误。然而更强的 ECC 校验能力需要数据携带更多的校验信息，这会带来一定程度的空间损耗。还有的方法提出可以从读重试的过程入手，通过准确预测读重试的最佳读参考电压来减少读重试次数<sup>[4,17]</sup>。增强 ECC 校验能力和减少读重试操作都是为了提高闪存的读干扰容忍度，但是都不能从根本上减轻读干扰。因此，部分读干扰缓解方案从数据管理方面入手，提出可

以将热读数据进行合理放置来帮助减轻闪存块的读干扰<sup>[20-21,79]</sup>。在这些相关工作中,为了缓解读干扰,许多方法都需要付出很大的代价(空间损耗或时间开销)。直观上来说,在闪存的 DRAM 中设立一个读缓存机制,将热读数据缓存到 DRAM 中会是缓解读干扰的一个有效手段。

在闪存内嵌的 DRAM 中设置读缓存这个方法存在两个难以解决的问题。第一个问题是,由于闪存系统中的写入方式是基于异地更新的,需要通过垃圾回收操作对一些失效数据进行擦除,而垃圾回收造成的写放大会严重影响闪存系统的性能,再加上闪存系统中的写入操作比读取操作的时间要长很多,因此相比于读取操作,写入操作对 NAND 闪存的影响更大,将 DRAM 空间分配给写缓存往往比分配给读缓存更有性价比;第二个问题是,即使设立了读缓存,但是由于闪存系统中的 DRAM 空间有限,往往不能将全部热数据都吸收到读缓存中,很难判断应该缓存哪些数据。针对这两个问题,本文提出了基于读干扰感知的数据缓存策略 Minato。Minato 在闪存内嵌 DRAM 中同时设立了写缓存和读缓存,通过检测写缓存的命中率变化情况,动态改变写缓存和读缓存的大小。在 Minato 中,为读缓存分配空间的原则是尽量不影响写缓存的命中率。此外,通过监测闪存块的读干扰情况,Minato 为读缓存设置了一个数据过滤器。当热数据较多且读缓存的空间不够时,优先缓存读干扰影响较大的数据(详见章节 4.2),从而降低 NAND 闪存整体的读干扰水平。Minato 的主要动机和可行性总结如下:(1) 虽然 NAND 闪存中写缓存比读缓存更为重要,但是在热读场景中,在不影响写缓存的情况下同样可以构建读缓存来为热读数据服务;(2) 读缓存的缓存能力有限,减轻有效数据较多的闪存块上的读干扰相对来说更有性价比;(3) Minato 读干扰感知缓存策略只需要获取到闪存块的读取次数和有效页面数量,而这些都可以在读取数据时知悉,策略可行性高。

由于闪存系统通常会将部分地址映射表还有部分热数据共同缓存到 DRAM 中,而且 DRAM 空间又比较有限,因此现有的关于缓存空间分配的策略大部分集中于如何在地址映射表和热数据 buffer 之间做取舍上<sup>[29-31]</sup>。Minato 与这些研究并不冲突,并且是在这些研究基础之上设计的新策略,与先前的策略可以叠加使用。现有的针对读干扰来设计缓存策略的研究较少,Minato 是专门为了缓解读干扰而设计的数据缓存策略。

综上所述,本文为缓解 NAND 闪存读干扰和提升 NAND 闪存性能做了以下三点贡献:

(1) 分析了读干扰对不同闪存块状态的影响程度,为量化读干扰的影响提供了一个新的角度;

(2) 为了不影响闪存系统的性能同时更好的缓存读热数据,提出了一种低成本高效率

的数据缓存管理方案 Minato。Minato 能够感知闪存块的状态和读干扰情况，从而更好的缓存热读数据，从总体上减轻闪存系统的读干扰。

(3) 用多个真实环境下的工作负载验证了 Minato。实验结果表明，Minato 可以在不影响写请求命中率的情况下缓解闪存的整体读干扰。与单纯的读缓存策略相比，Minato 的整体读干扰水平、读重写次数、读反应时间最高可减少 3.68%、15.32%、13.14%。

## 4.2 NAND 闪存读干扰影响情况分析

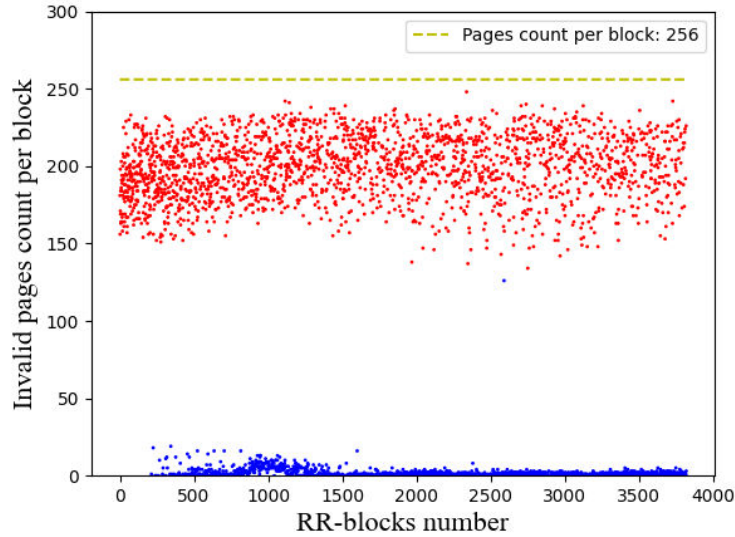


图 4.1 严重读干扰引发读重写时的闪存块页面失效情况

为了理解真实环境工作负载的数据访问行为，实验部分获取到微软、亚马逊、阿里巴巴等公司发布的多个真实环境下闪存系统工作负载。在相同实验条件（详见 4.4 章节的实验设置）下测试了不同工作负载的数据特征。图 4.1 是阿里巴巴在 2020 年发布的负载数据<sup>[70]</sup>中一个读密集型工作负载测试出来的实验结果。RR-blocks 是由于严重读干扰而引发擦除操作（读重写）的闪存块。横坐标表示的是工作负载运行过程中发生的读重写次序（如第  $x$  个发生读重写的闪存块）；纵坐标表示的是闪存块的失效页面数量，实验中闪存块的总页面数量设定为 256，并且将失效页面超过闪存块总页面数量一半的部分标记为红色，不超过总页面数量一半的部分标记为蓝色。图中表示的是严重读干扰引发读重写时的闪存块页面失效情况，从图中可以了解到，部分闪存块在遭受严重读干扰并且发生读重写时，闪存块中已经存在大量的失效页面；而有的闪存块在遭受严重读干扰时块中基本都还是有效页面。由此提出可以从一个新的角度来缓存热读数据。

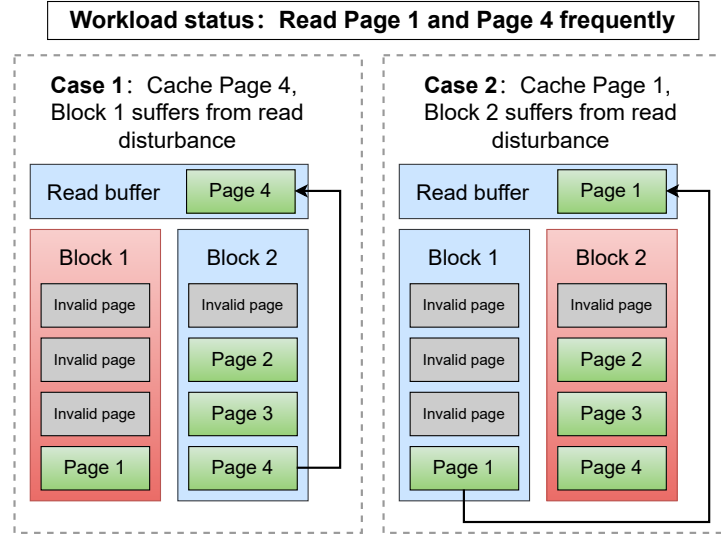


图 4.2 不同缓存策略对闪存读干扰的影响情况

图 4.2 中展示了不同缓存策略对闪存块读干扰的影响。假设当前有两个闪存块 Block1 和 Block2，闪存块容量为 4 个页面；闪存块 Block1 中三个页面已经失效，只有 Page1 页面有效；闪存块 Block2 中只有一个失效页面，Page2、Page3、Page4 都是有效页面；存在一个读缓存，读缓存大小只能容纳一个页面。假设当前的工作负载会频繁地读取页面 Page1 和页面 Page4，且该读取频率都足以对闪存块造成严重读干扰。在 Case1 中，读缓存接收的是 Page4，经过频繁读取后由于 Page1 没有进入缓存，因此所在的 Block1 遭受了严重读干扰；在 Case2 中，读缓存接收的是 Page1，经过频繁读取后 Block2 遭受了严重读干扰。在这两个情况中，假设 Case1 的 Block1 和 Case2 的 Block2 遭受读干扰的程度一致，由于 Block1 只有一个有效页面，因此读干扰只会影响到 Page1，而 Block2 中的读干扰则会同时影响到其中的三个页面。在闪存系统中，读干扰会影响电压分布从而读取数据时需要比以往更长的读取时间。Block2 中的读干扰不仅会影响到频繁读取的 Page4，同时还会影响到可能与 Page4 无任何关联的其它进程的数据 Page2 和 Page3。因此，在图 4.2 中的这两种情况，缓存有效页面更多的 Block2 中的热读数据明显比缓存有效页面少的 Block1 中的热读数据更有性价比。

基于以上观察和发现，本文提出了基于读干扰感知的数据缓存策略 Minato。Minato 中存在一个数据过滤机制，在读缓存空间十分有限且热读数据较多的情况下，通过数据过滤机制，只缓存读干扰影响较大的数据，也即是有效页面更多的闪存块中的热读数据。另外，由于存在垃圾回收机制，有效页面较少的闪存块会更容易被擦除，从而在严重读干扰状态中恢复过来；相反，有效页面较多的闪存块则可能需要经历更长的读写周期，读干扰

伴随数据的时间也会更长，延长了读反应时间甚至引发读重写。因此，通过数据缓存策略 Minato 来缓解一部分闪存块的读干扰是十分必要的。之后将在 4.3 章节中具体描述 Minato 的设计思路和实现细节。

## 4.3 Minato 的设计与实现

### 4.3.1 Minato 的设计概述

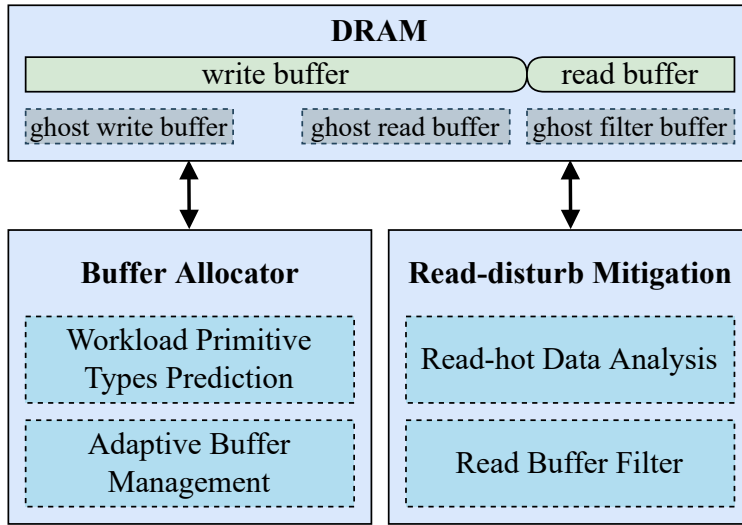


图 4.3 Minato 结构总览

如图 4.3 所示, Minato 的设计主要分为两个部分, 分别是读写缓存空间分配机制 (Buffer Allocator) 和读干扰缓解机制 (Read-disturb Mitigation)。Minato 提倡在不影响写缓存的情况下设立读缓存, 因此在空间总量不变的情况下, DRAM 中写缓存和读缓存的大小是动态分配的。DRAM 中还同时设立了虚写缓存 (ghost write buffer)、虚读缓存 (ghost read buffer) 和虚过滤缓存 (ghost filter buffer), 写缓存中被替换掉的闪存页面信息不会立即删除, 而是先保存到虚写缓存中 (虚写缓存不保存数据页面, 只保存数据信息), 同样的读缓存被替换的页面信息是先保存在虚读缓存中, 被读干扰缓解机制过滤掉的读请求信息会先保存到虚过滤缓存中。由于虚缓存中保存的都是刚从缓存中替换出来的数据信息, 当数据没有在缓存中命中时, 通过查看其是否存在虚缓存中可以帮助分析当前的数据流情况。

在读写缓存空间分配机制中存在两个模块, 分别是负载基本类型预测 (Workload Primitive Types Prediction) 和自适应缓存管理 (Adaptive Buffer Management)。Minato 可以根据滑动回望窗口 (sliding look-back window) 的请求服务情况, 判断当前负载的基本类型, 计算读写比例和缓存命中率。Minato 会基于计算的结果, 再结合当前虚缓存的命中率, 判断

是否应该对缓存空间做出调整。由于闪存系统中缓存写请求的性价比更高，因此在自适应缓存管理中会给予写缓存更高的优先级。具体的细节将在 4.3.2 章节中阐述。

在读干扰缓解机制中设置了热读数据分析模块（Read-hot Data Analysis）和读缓存数据过滤模块（Read Buffer Filter）。Minato 会根据当前虚读缓存的命中率以及读缓存的空间大小来判断热读数据情况。假如虚缓存被频繁命中，则证明当前热数据较多，可以选择性缓存读干扰影响较大的数据。在 Minato 中存在一个过滤因子，通过调整过滤因子可以调整过滤的数据范围。过滤掉的数据可能也存在一些热读数据，因此 Minato 同时还会维持一个虚过滤缓存，将过滤掉的数据信息保存到虚过滤缓存中，并通过一个先进先出链表来维系它们。假如链表中不存在该数据信息，则直接将数据信息放入表中；假如数据信息存在链表中，则 Minato 认为该数据是相对频繁的热数据，可以绕过过滤器进入缓存。具体的细节将在 4.3.3 章节中阐述。

#### 4.3.2 读写缓存空间分配机制

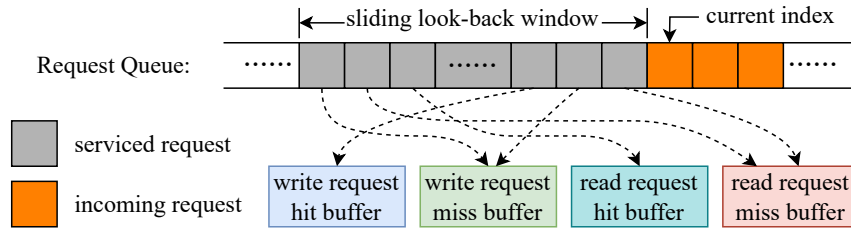


图 4.4 滑动回望窗口的请求服务情况

如图 4.4 所示，Minato 中设置了一个滑动回望窗口。这个窗口会记录过去已服务完成的  $n$  个请求的信息，比如分属于读写哪种请求、缓存命中情况等， $n$  值的大小与 buffer 的大小一致。通过滑动回望窗口的请求信息，可以分析负载的基本类型（读多写少、写多读少、读写均衡），计算当前的读写比例和缓存命中率。将滑动回望窗口中的写请求数量用  $write\_count$  表示；写请求的缓存命中率用  $write\_hit$  表示；读请求的数量用  $read\_count$  表示；读请求的缓存命中率用  $read\_hit$  表示。假设以读写反应时间为基准，由于写反应时间通常为读反应时间的十倍，可以将写请求的权重设置为 10，将读请求的权重设置为 1。当以下关系式成立时，则满足减少写缓存空间、增大读缓存空间的前提：

$$write\_count * write\_hit < read\_count / 10 * read\_hit, \quad (4.1)$$

滑动回望窗口的信息统计方式定义如下：滑动回望窗口的大小  $n$  和缓存空间大小一致，并且每次关系式计算之后的结果会应用到之后的  $n$  个请求中，也就是说，每隔  $n$  个请求才



会统计一次请求信息。假如统计结果不满足关系式，则在之后的  $n$  个请求期间缓存空间的比例关系都不再改变；假如满足该关系式，则在之后的  $n$  个请求期间缓存空间的比例关系也只可能改变一次。以此类推，之后的  $n$  个请求的计算结果将影响它接下来的  $n$  个请求。通过对当前回望窗口中的信息进行统计可以判断当前负载的基本情况，为后续操作提供先验知识。另外，相比于每个请求到来时都去统计回望窗口的信息，每隔  $n$  个请求统计一次的计算成本会更低一些；而且滑动回望窗口的滑动幅度较小时统计结果没有明显差异，每隔  $n$  个请求统计一次的信息价值更大。

值得注意的是，满足以上关系式只是作为缩小写缓存空间的前提，具体操作时还需要考虑虚写缓存的命中率。假如虚写缓存在记录滑动回望窗口期间有过命中，则不缩小写缓存空间，相反地，缩小读缓存空间并增大写缓存空间。Minato 的读写缓存空间分配分为初始阶段和平衡阶段。具体细节如下：

#### 初始阶段：

在初始阶段，缓存空间一开始全都分配给写缓存，之后每一个请求到来时再重新规划缓存空间。如算法 3 所示，当请求数据不在缓存空间中时，首先判断 DRAM 空间是否已满，假如已满则结束初始阶段，进入平衡阶段；假如未满则判断到来的是否是写请求，如果是则将写请求数据放入写缓存，且不改变读缓存和写缓存的空间划分；如果不是，则将写缓存的空间减一、读缓存的空间加一，并将读请求数据放入读缓存中。

---

#### Algorithm 3 初始阶段缓存空间分配策略

---

```

1: /*At the beginning, write buffer has all the DRAM space*/
2: if buffer is not full then
3:     if the request data exists in buffer then
4:         update the cached data information
5:     else
6:         if request is a write request then
7:             put request data to write buffer
8:         else
9:             write buffer space reduced by one
10:            read buffer space plus one
11:            put request data to read buffer
12:        end if
13:    end if

```



---

```

14: else
15:     go to the equilibrium stage
16: end if

```

---

### 平衡阶段：

平衡阶段下的 DRAM 空间已满，且写缓存和读缓存各占其中一部分空间。由于工作负载中写请求和读请求的密集程度和比例关系会一直随时间变化，因此 Minato 需要不断对写缓存和读缓存的空间进行调整。又因为缓存写请求的优先级更高，在制定调整策略时会更倾向于提高写缓存的命中率。Minato 为写缓存维护一个虚写缓存，虚写缓存的长度和读缓存的大小保持一致，当数据从写缓存空间中淘汰后，数据信息会放置到虚写缓存中。虚写缓存采用 LRU 的替换策略。

如算法 4 所示，在平衡阶段，当请求数据不在缓存空间中时，Minato 首先判断请求类型，如果是读请求，则进入处理读请求的阶段（详见章节 4.3.3）；如果是写请求，则进入下一个判断流程。下一步中 Minato 会判断写请求数据信息是否存在于虚写缓存中，如果存在于虚写缓存中，则意味着写缓存空间不足，此时将写缓存空间加一，读缓存空间减一，并将写请求数据进入写缓存；如果不存在于虚写缓存中，则进入下一个判断流程。接下来 Minato 会判断滑动回望窗口中的计算结果是否满足关系式 4.1。Minato 的滑动回望窗口每隔  $n$  个请求滑动一次，关系式同样每隔  $n$  个请求（ $n$  为缓存空间大小）计算一次；Minato 缓存空间的分配情况在同一个滑动回望窗口期间只会改变一次。如果关系式 4.1 成立且在该滑动回望窗口期间缓存空间还没有发生改变，则写缓存空间减一，读缓存空间加一，并将写请求数据放入写缓存；否则不改变空间分配情况，直接将写请求数据放入写缓存。

---

#### Algorithm 4 平衡阶段缓存空间分配策略

---

```

1: if request is a write request then
2:     if the request data exists in buffer then
3:         update the cached data information
4:     else
5:         if the request information exists in ghost write buffer then
6:             /*read buffer space != 0*/
7:             write buffer space plus one
8:             read buffer space reduced by one
9:         else
10:            if  $write\_count * write\_hit < read\_count / 10 * read\_hit$  then

```

---

```

11:          /*write buffer space != 0*/
12:          read buffer space plus one
13:          write buffer space reduced by one
14:      end if
15:      put the request data to write buffer
16:      put the evicted data information of write buffer into ghost write buffer
17:  end if
18: end if
19: else
20:     handle read requests (explained in next section)
21: end if

```

---

#### 4.3.3 读干扰缓解机制

由于缓存空间相对于闪存系统容量来说要小得多，当频繁访问的数据量较大时缓存空间往往不足以容纳所有热数据。从章节 4.2 中的分析可以得知，相同程度的读干扰对有效页面较多的闪存块的影响会比有效页面较少的闪存块的影响更大。当热数据较多时，适当过滤有效页面较少的闪存块中的数据，给有效页面较多的闪存块中的数据更多机会进入缓存，可以在一定程度上减少读干扰的影响。同时，Minato 会保存一段时间内被过滤掉的有效页面较少的闪存块中的数据信息，争取将被过滤的数据中的热数据吸纳进缓存。以下是 Minato 读干扰缓解机制的详细介绍。

图 4.5 展示了数据分布可能出现的两种情况。假设读缓存具有两个页面位置，闪存中有四个闪存块。情况 1 中的 Block1 和 Block3 只有一个有效页面，但是都是热读页面；Block2 和 Block4 都有三个有效页面，其中的页面都是冷读页面，由于读缓存中只有两个位置，因此优先将 Block1 和 Block3 中的热读页面存入缓存中。情况 2 中的 Block1 和 Block3 与情况 1 一样都具有一个热读有效页面，Block2 和 Block4 则分别具有两个冷读页面和一个热读页面，此时根据 4.2 章节中的读干扰影响情况分析，应该缓存 Block2 和 Block4 中的热读页面 Page4 和 Page8。根据上述分析，本文提出当热读数据较少时，不管数据页面位于什么物理位置都要对其进行缓存，即过滤机制应该对请求数据保持透明；而当热读数据较多时，过滤机制可以发挥作用，只缓存读干扰影响较大的闪存页面。

Minato 用 *valid\_proportion* 来表示闪存块中有效页面的比例，用 *filter\_x* 来表示过滤机制的过滤因子。*filter\_x* 的意义是，过滤掉所在闪存块有效页面比例 *valid\_proportion* 小于

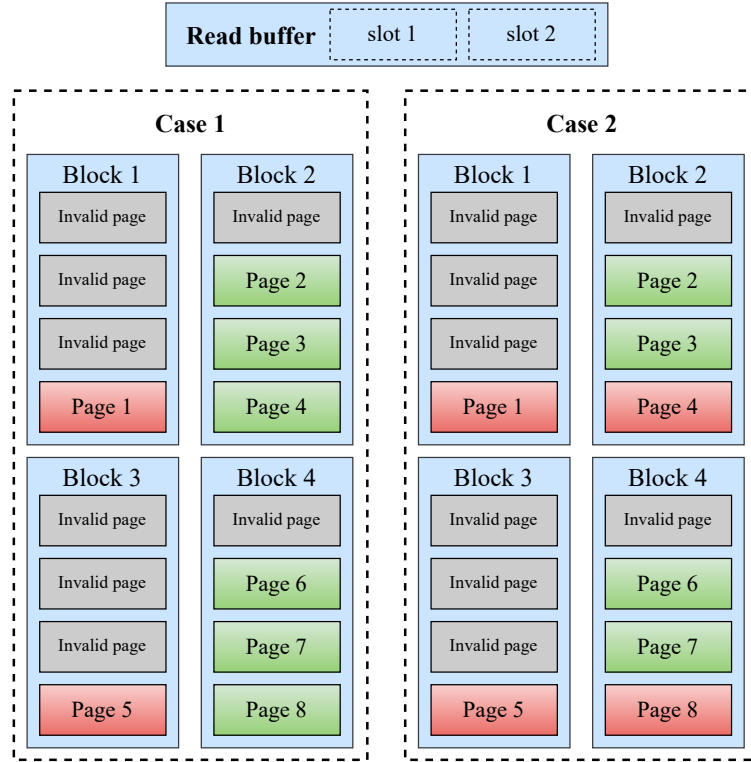


图 4.5 数据分布的两种情况

$filter\_x$  的数据。Minato 在 DRAM 设置了一个虚读缓存来帮助评估热数据，虚读缓存的大小设置为与读缓存的大小一致。另外，Minato 还设置了一个虚过滤缓存来存放过滤掉的部分数据，虚过滤缓存的大小与读缓存大小一致。

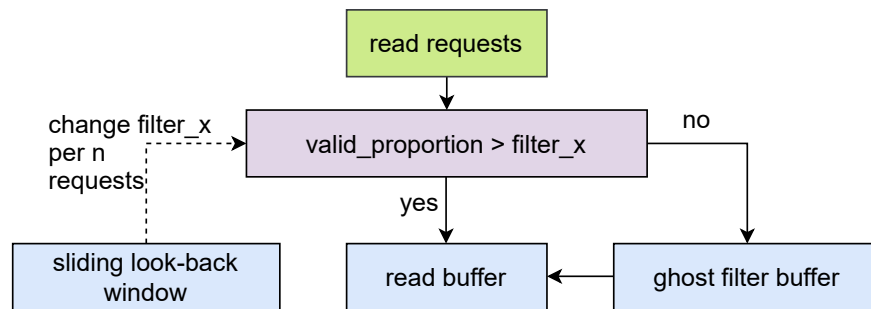


图 4.6 读缓存过滤机制

图 4.6 展示了读缓存的过滤机制。初始阶段时， $filter\_x$  的大小设置为 0，即不会过滤任何数据。在章节 4.3.2 中提到 Minato 会维持一个滑动回望窗口，在滑动回望窗口中，Minato 会同时记录下读请求命中虚读缓存的情况。Minato 的滑动回望窗口每隔  $n$  个请求滑动一次，虚读缓存的命中情况同样每隔  $n$  个请求查看一次。当虚读缓存没有被命中时，可以认为当前的读缓存大小足以应付该程度的热数据请求，所有闪存页面数据都可以进入缓存，不做限制，此时如果  $filter\_x$  的大小不为 0，则将其重置为 0。当虚读缓存被命中时，意味着当

前读缓存空间不足, Minato 会对读请求进行过滤, 即将  $filter\_x$  的值缓慢增大, 且在同一个滑动回望窗口中只会增大一次。由于热读数据量的不确定性, 过滤读请求的比例变化需要尽可能缓慢, Minato 将每次增大的比例设置为 0.1。

---

**Algorithm 5** 读请求缓存策略
 

---

```

1: /*For read requests*/
2: if  $read\_requests\_count++ == \text{buffer size}$  then
3:   adjust  $filter\_x$ 
4:    $read\_requests\_count = 0$ 
5: end if
6: if the request data exists in read buffer then
7:   update the cached data information
8: else
9:   if  $valid\_proportion > filter\_x$  then
10:    put the evicted data information of read buffer into ghost read buffer
11:    put the request data to read buffer
12:  else
13:    if the request information exists in ghost filter buffer then
14:      put the request data to read buffer
15:    else
16:      put the request information to ghost filter buffer
17:    end if
18:  end if
19: end if

```

---

读缓存经过过滤后, 有效页面较多的闪存块中的数据在缓存中的占比可能会增大, 但是由于替换策略没有发生改变, 还是会有相当多的冷数据进入缓存进行缓存争用。读缓存过滤给了有效页面较多的闪存块中的数据更多的访问缓存的机会, 但是并没有从本质上提高缓存命中率。因此, 在数据过滤的基础上, Minato 增加了一个虚过滤缓存机制。由于过滤掉的数据可能存在一些热数据, 这些数据所在的闪存块虽然有效页面较少, 但是访问频率很高。Minato 中设置了一个虚过滤缓存模块, 过滤掉的数据信息会先进入这个模块。只要读请求在虚过滤缓存中命中, 则 Minato 直接让该数据进入读缓存。通过这种方式, Minato

给有效页面较多的闪存块中的数据更多的缓存机会，同时在缓存中留住了有效页面较少的闪存块中的热数据。具体流程如算法 5 所示。

## 4.4 实验验证及分析

### 4.4.1 实验设置

本文在模拟器 MQSim<sup>[77]</sup>中实现了 Minato。MQSim 通过模拟闪存控制器的不同事件及其连锁反应，能够精确模拟现代 NAND 闪存的评估指标，比如缓存命中率、读写请求反应时间等。Minato 在源码的基础上添加读重写机制，设置读重写可以根据闪存块的读次数进行触发，并且将每个闪存块触发读重写的阈值设置为页面数量的 40 倍<sup>[11]</sup>。表 4.1 总结了 MQSim 模拟器的配置细节。实验中将一个闪存页面的大小设置为 8KB，256 个闪存页面组成一个闪存块。MQSim 模拟了真实闪存系统对每个闪存块的读次数进行计数的细节，在后续的实验，闪存页面所处闪存块的读次数可以用来衡量页面的读干扰情况。延迟方面，模拟器中延续了 MQSim 本身的设置，将读延迟设置为 75 微秒、写延迟设置为 750 微秒、擦除延迟设置为 3.8 毫秒。另外，由于后续使用的基本都是企业级工作负载，因此将 GC 的触发阈值设置为 0.28。

表 4.1 NAND 闪存模拟器实验配置

|                         |                         |
|-------------------------|-------------------------|
| SSD organization        | 8 channels              |
|                         | 4 chips per channel     |
|                         | 2 dies per chip         |
| Flash Microarchitecture | 2 planes per die        |
|                         | 2048 blocks per plane   |
|                         | 256 pages per block     |
|                         | Page size: 8 KB         |
| Flash Latencies         | Read latency: 75 us     |
|                         | Program latency: 750 us |
|                         | Erase latency: 3.8 ms   |
| GC Parameters           | Policy: GREEDY          |
|                         | Threshold: 0.28         |

本文在九个不同读写比例的真实工作负载上进行实验（见表 4.2），其中 ali1、ali2 和 ali3 是阿里巴巴某个数据中心三台虚拟机在 2020 年其中一个月的访问行为<sup>[70]</sup>。prn、proj、src1 和 src2 都来自于公开的磁盘工作负载 MSR Cambridge traces<sup>[74]</sup>。systor 记录了企业虚拟桌面基础设施在 2017 年某一个月的数据访问行为<sup>[71]</sup>。usr1 来自企业集群<sup>[74]</sup>。表 4.2 展示

了这九个不同工作负载的读写请求情况，并将它们按读请求比例的大小进行排列。从表中可以看到，这些工作负载基本覆盖了所有读写场景，既有读请求比例较低的工作负载 *prn*，也有读请求比例较高的工作负载 *src2*，通过模拟运行这些工作负载可以对 Minato 的多场景普适性进行验证。实验结果将在后续章节进行展示和说明。

表 4.2 工作负载的读写请求情况

| Trace         | trace size(GB) | read ratio | reads count | writes count |
|---------------|----------------|------------|-------------|--------------|
| <i>prn</i>    | 59             | 0.1079     | 6.03e5      | 4.98e6       |
| <i>ali2</i>   | 76             | 0.4721     | 1.53e6      | 1.72e6       |
| <i>src1</i>   | 1538           | 0.5643     | 2.11e7      | 1.63e7       |
| <i>systor</i> | 312            | 0.5968     | 1.02e7      | 6.90e6       |
| <i>ali3</i>   | 1426           | 0.6499     | 4.07e7      | 2.19e7       |
| <i>ali1</i>   | 1692           | 0.6948     | 3.21e7      | 1.41e7       |
| <i>usr</i>    | 441            | 0.8113     | 8.58e6      | 1.99e6       |
| <i>proj</i>   | 1184           | 0.8761     | 2.56e7      | 3.62e6       |
| <i>src2</i>   | 1516           | 0.9526     | 4.36e7      | 2.17e6       |

#### 4.4.2 实验介绍

Minato 主要由两部分组成，分别是读写缓存空间分配机制和读干扰缓解机制。因此，实验部分将分别针对这两个部分的情况进行分析。第一部分，通过将 Minato 与另外两种缓存空间分配策略进行比较，可以评估 Minato 在写缓存命中率方面的影响；第二部分，将缓存空间都分配给读缓存，并将缓存空间梯度递增，可以评估 Minato 读干扰缓解机制的有效性。

读写缓存空间分配机制最重要的衡量指标是对写请求命中率的影响。由于闪存系统中相比于缓存读请求，缓存写请求的性价比往往更高，因此在做读写缓存空间分配时需要尽可能不影响写请求的缓存命中率。实验的第一部分通过将 Minato 与其它两种不同的缓存空间分配策略进行比较来评估其性能，分别是只设置写缓存（Only-write）和读写分离缓存（WR+RR）<sup>[31]</sup>。在 Only-write 中，所有读请求不经过缓存空间，直接到闪存中进行读取，DRAM 中的缓存空间全部分配给写缓存。WR+RR 是 yeh 等人<sup>[31]</sup>提出的一种读写分离缓存策略。与 Minato 类似，在 WR+RR 中，读缓存和写缓存的空间会根据当前读写请求的频率和热度动态变化。读请求数据和写请求数据在 DRAM 缓存空间中的替换策略彼此独立，WR+RR 通过维护两个链表分别组织读请求数据和写请求数据。

在读写缓存空间分配机制之上还有一个读干扰缓解机制。读干扰缓解机制是基于读缓

存设置的, 不涉及写缓存替换策略, 因此本文在第二个实验部分单独设置读缓存进行实验, 避免其它因素影响读干扰缓解机制有效性的验证。Minato 在读缓存基础上添加了一个请求过滤机制, 根据当前工作负载的情况, 适当地让有效页面更多、读干扰影响更大的闪存块中的数据有更多机会进入缓存。对于有效页面较少的闪存块中的数据, 读干扰缓解机制通过设置虚过滤缓存, 让较热的数据同样有机会进入缓存。具体实验流程如下: 在仅设置读缓存的条件下, 将单纯的读缓存和加了读干扰缓解机制的读缓存进行对比; 将读缓存空间从 1MB 到 16MB 递增, 观察读重写次数、整体读干扰水平和读反应时间的变化, 从而对读干扰缓解机制的性能进行评估。

#### 4.4.3 写请求命中率情况分析

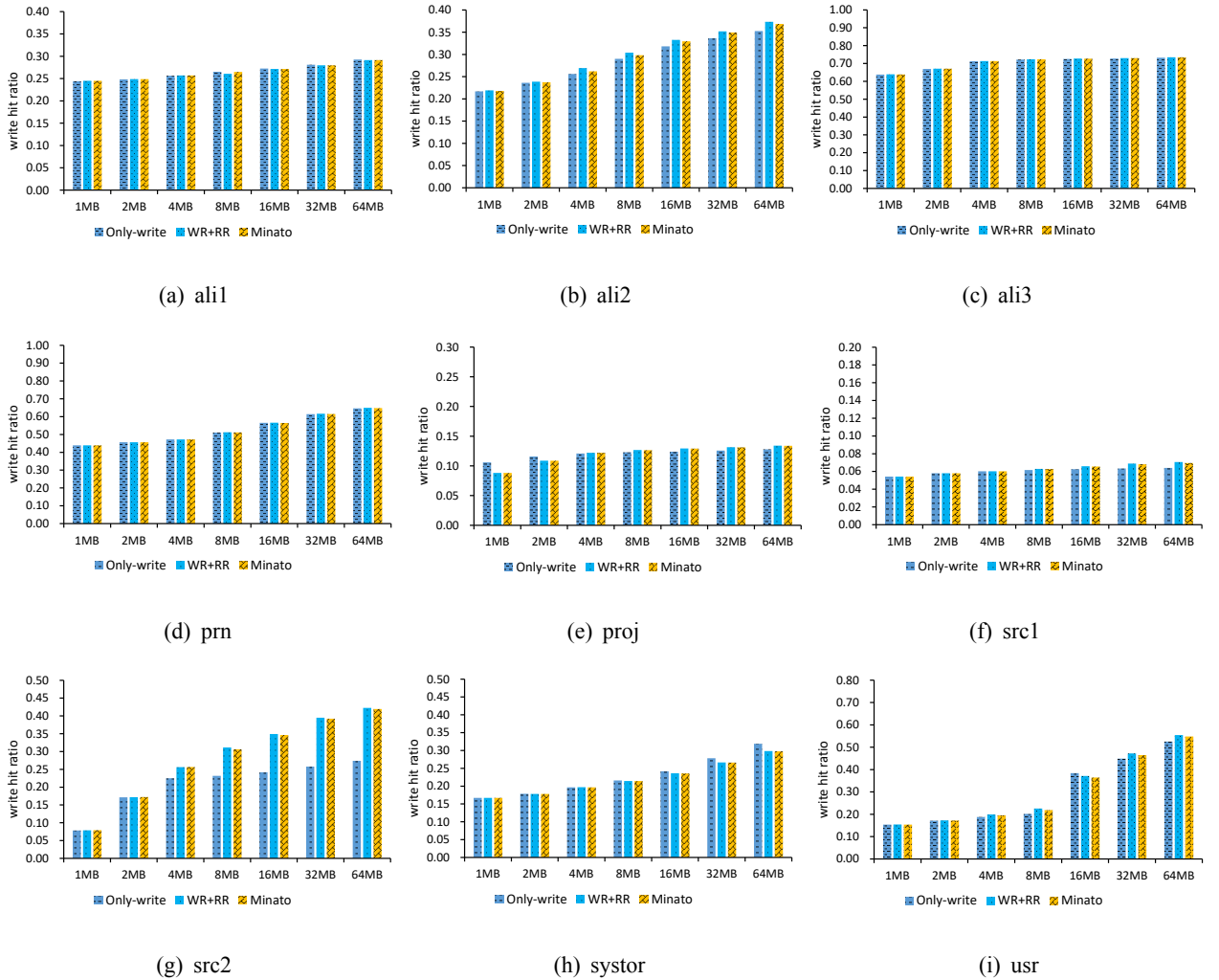


图 4.7 写请求命中率情况

在这一章节中主要分析 Minato 对闪存写请求命中率的影响。通过将 Minato 与其它两种不同的缓存策略进行比较来评估 Minato 的写请求命中率表现, 分别是只设置写缓存

(Only-write) 和读写分离缓存 (WR+RR)<sup>[31]</sup>。写请求进入缓存空间的流程如下：当写请求到来时，假如设置了写缓存，则写请求先到写缓存空间中寻找对应的逻辑地址，找到了则更改缓存数据信息，没有找到则到读缓存空间中进行寻找，假如读缓存也没有对应的数据，则视为缓存不命中。一般而言，如果设置了读缓存（比如 WR+RR 和 Minato），写缓存需要将一部分缓存空间让给读缓存，写缓存空间的减小可能会降低写请求的命中率。考虑到这个问题，Minato 中的读写缓存空间分配策略设置为动态的，即 Minato 会通过缩小读缓存空间来优先考虑写请求的命中率，尽量减少加入读缓存之后对写请求命中率的影响。

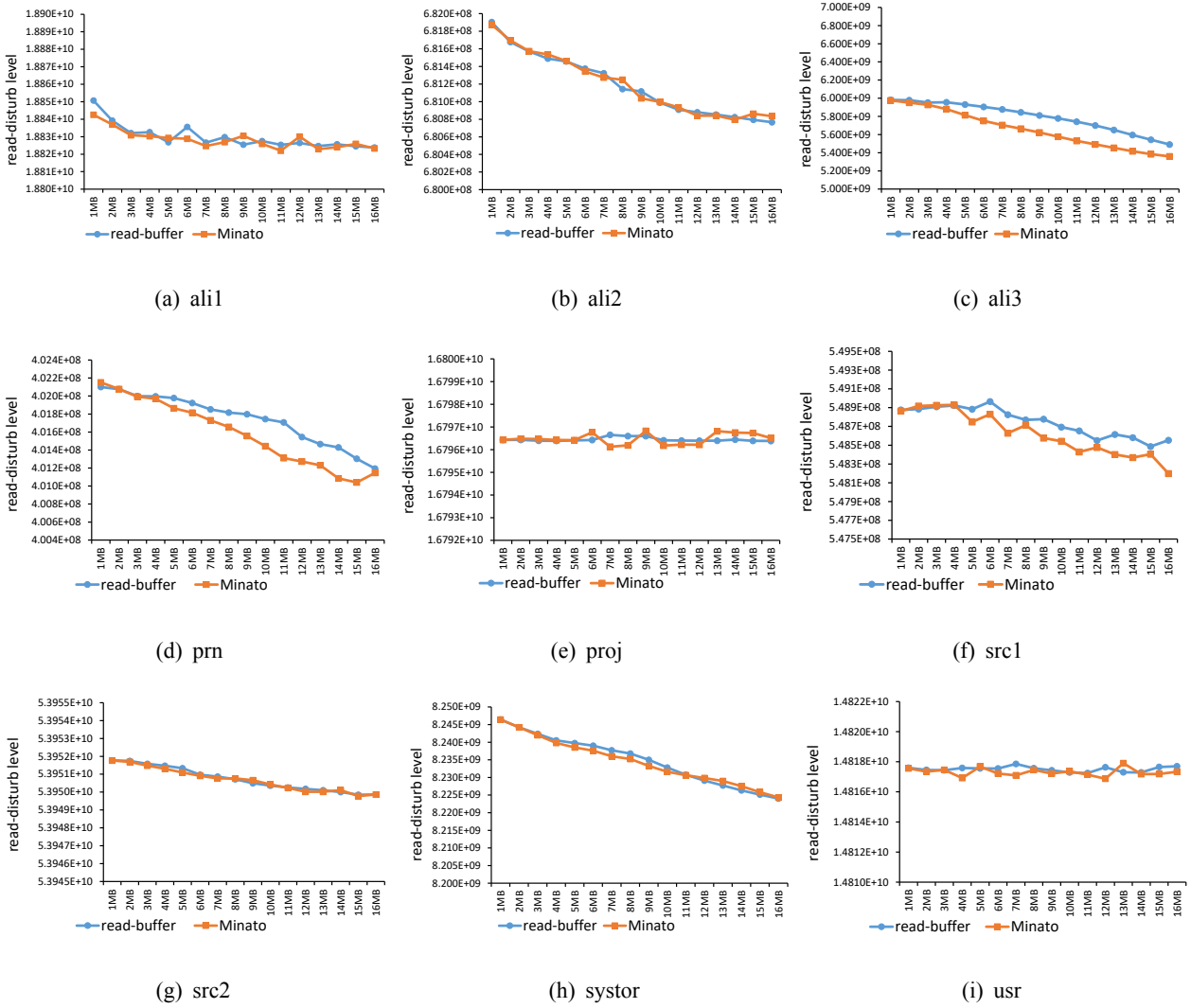


图 4.8 整体读干扰情况

如图 4.7 所示，实验中设置了七组不同大小的缓存空间，分别为 1MB、2MB、4MB、8MB、16MB、32MB 和 64MB。横坐标为不同缓存空间大小，纵坐标为写请求命中率。可以看到，在大多数工作负载中，由于读写缓存空间分配策略的作用，Minato 和 WR+RR 虽然增加了读缓存，但是并不会对写请求的命中率造成严重影响。如实验结果所示，三种策



略在 ali1、ali3、prn 这三个工作负载中的写请求命中率基本一致。并且，由于写请求进入缓存空间时会搜寻读缓存中的数据，假如部分数据的读取和写入频率较为一致，读取之后马上做数据更新，则写请求数据更有可能在读缓存中命中。加上了读缓存反而能提高写请求的命中率。由于 ali2、proj、src1 和 src2 工作负载中部分数据的读取和更新频率较为一致，Minato 和 WR+RR 甚至有比 Only-write 更好的缓存命中率。在 systor 工作负载中，读缓存的加入稍微降低了写请求的命中率；在 usr 工作负载中，16MB 缓存空间大小下读缓存的加入稍微降低了写请求命中率，而在其它缓存空间大小下则对写请求命中率有细微的提升。

综上所述，Minato 的读写缓存空间分配策略并不会影响以上工作负载的写请求命中率，在某些工作负载上甚至有更优表现。

#### 4.4.4 闪存整体读干扰情况分析

在这一章节中主要分析 Minato 对闪存整体读干扰的缓解情况。

由于闪存的品类、制造工艺等各不相同，在闪存模拟器中很难将闪存读干扰精确量化。在先前的许多相关研究<sup>[15,18,80-82]</sup>中都指出，闪存页面的频繁读取会对整个闪存块上的数据都造成一定程度的读干扰影响。因此，在闪存模拟器中可以将每个闪存页面的读干扰量化为页面所处闪存块的总读取次数。例如，一个闪存页面当前所处闪存块的总读取次数为 1000，则这个闪存页面的读干扰系数就是 1000，也即该闪存块中所有有效页面的读干扰系数都是 1000。在实验中，每次工作负载进行结束的时候会计算当前闪存系统中所有页面的读干扰情况，并将所有页面的读干扰系数相加来反映闪存系统的整体读干扰情况。

为了更好的反映 Minato 读干扰缓解策略对读干扰的缓解程度，接下来的实验将在不设置写缓存，仅设置读缓存的条件下进行。目前已知在当前阶段还没有基于读干扰而制定的缓存方案，因此仅将 Minato 和单纯的读缓存机制 read-cache 进行对比。如图 4.8 所示，实验中设置了十六组不同大小的缓存空间，分别为 1MB、2MB、3MB、4MB、5MB、6MB、7MB、8MB、9MB、10MB、11MB、12MB、13MB、14MB、15MB 和 16MB。横坐标为不同缓存空间大小，纵坐标为整体读干扰系数。可以看到，由于缓存空间增大后可以吸纳的读请求更多，每个工作负载的读干扰随着缓存空间的增大都会减小。从实验结果中可以看到，虽然由于缓存空间太小对读干扰的吸收较少，但是在 ali2、ali3、prn、src1 这些工作负载中，Minato 都有更好的读干扰缓解作用。在 ali1 工作负载中，大部分情况下 Minato 都能更好地缓解读干扰，但是表现不稳定，在 5MB、9MB、12MB 的条件下没有缓解效果。在其它工作负载中的表现则与普通读缓存情况一致。这是因为 Minato 读干扰缓解机制的核

心在于，当热数据较多时，会给有效页面更多、读干扰影响更大的闪存块中的数据更多的机会进入缓存。假如工作负载的读请求基本都是冷数据或者说缓存空间大小足以应付热数据，则不会触发 Minato 的读干扰缓解机制，Minato 不会做额外的数据过滤操作，也就不会有更好的读干扰缓解效果。

从实验结果可得，与单纯读缓存情况相比，Minato 可以平均将九个工作负载的整体读干扰降低 0.44%，最高可以降低 3.68% (ali3)。

#### 4.4.5 闪存读重写情况分析

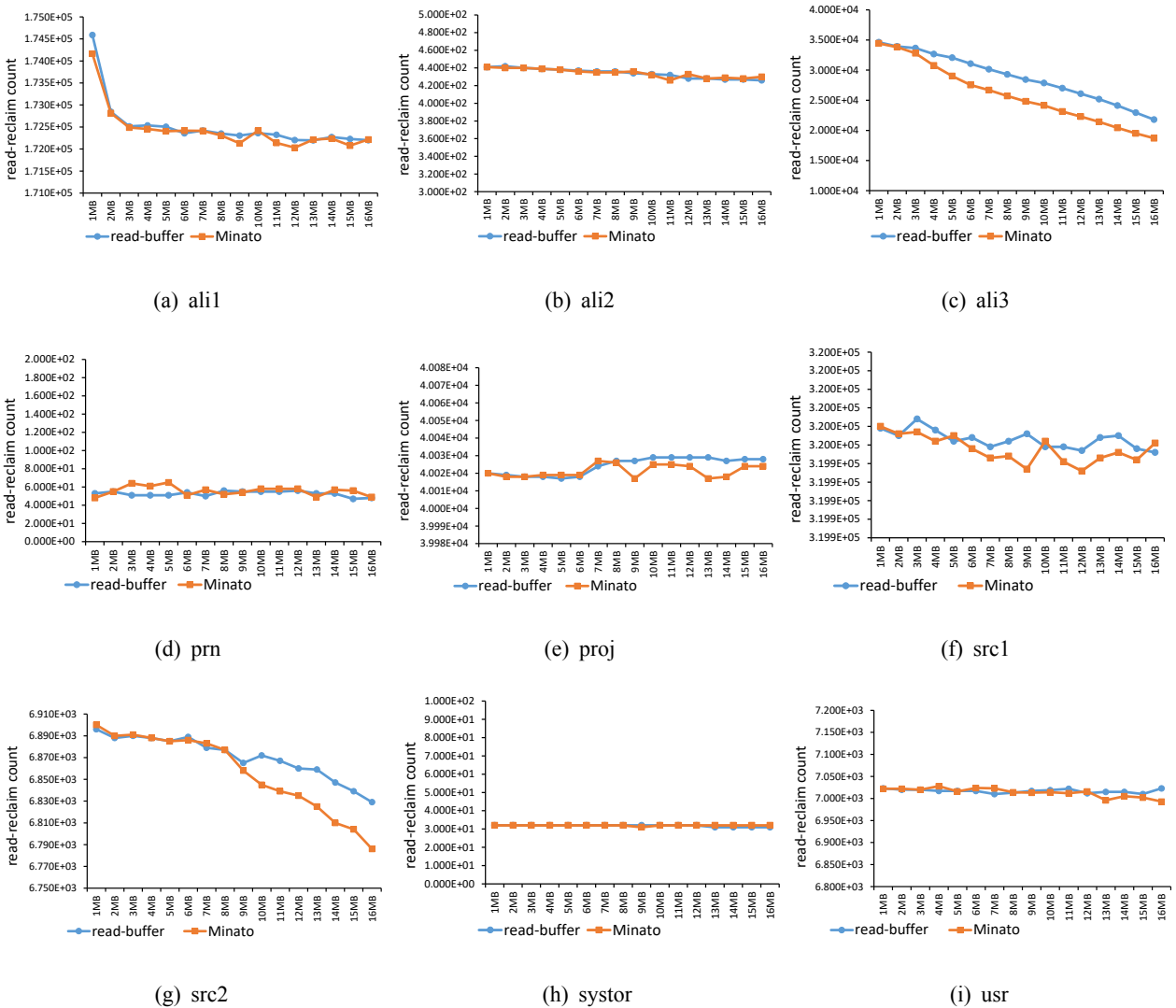


图 4.9 读重写触发情况

读重写的触发条件较为苛刻，在实验中将每个闪存块触发读重写的阈值设置为页面数量的 40 倍。本文采用的工作负载大多是服务器短时间（比如一个月）内的运行情况，产生的数据量无法触发读重写。为了模拟现实场景中持续不断的运行数据，实验部分将所有工

作负载在模拟器中重复运行十次。在读干扰情况分析中可知, Minato 可以一定程度地降低闪存整体读干扰。读重写是由于闪存块严重读干扰而触发的, 因此 Minato 同样可以一定程度地降低闪存的读重写次数。图 4.9 展示了两种读缓存策略触发读重写的情况。虽然由于缓存空间太小、对读请求的吸收较少导致实验没有非常明显的效果, 但是从图中还是可以看出在部分工作负载中 Minato 能更好的减少读重写的触发次数。其中, ali3 的实验效果最优, 比单独的读缓存降低了 15.32% 的读重写次数。

从实验结果可得, 与单纯读缓存情况相比, Minato 可以平均将九个工作负载的读重写次数降低 0.34%, 最高可以降低 15.32% (ali3)。

#### 4.4.6 读请求反应时间情况分析

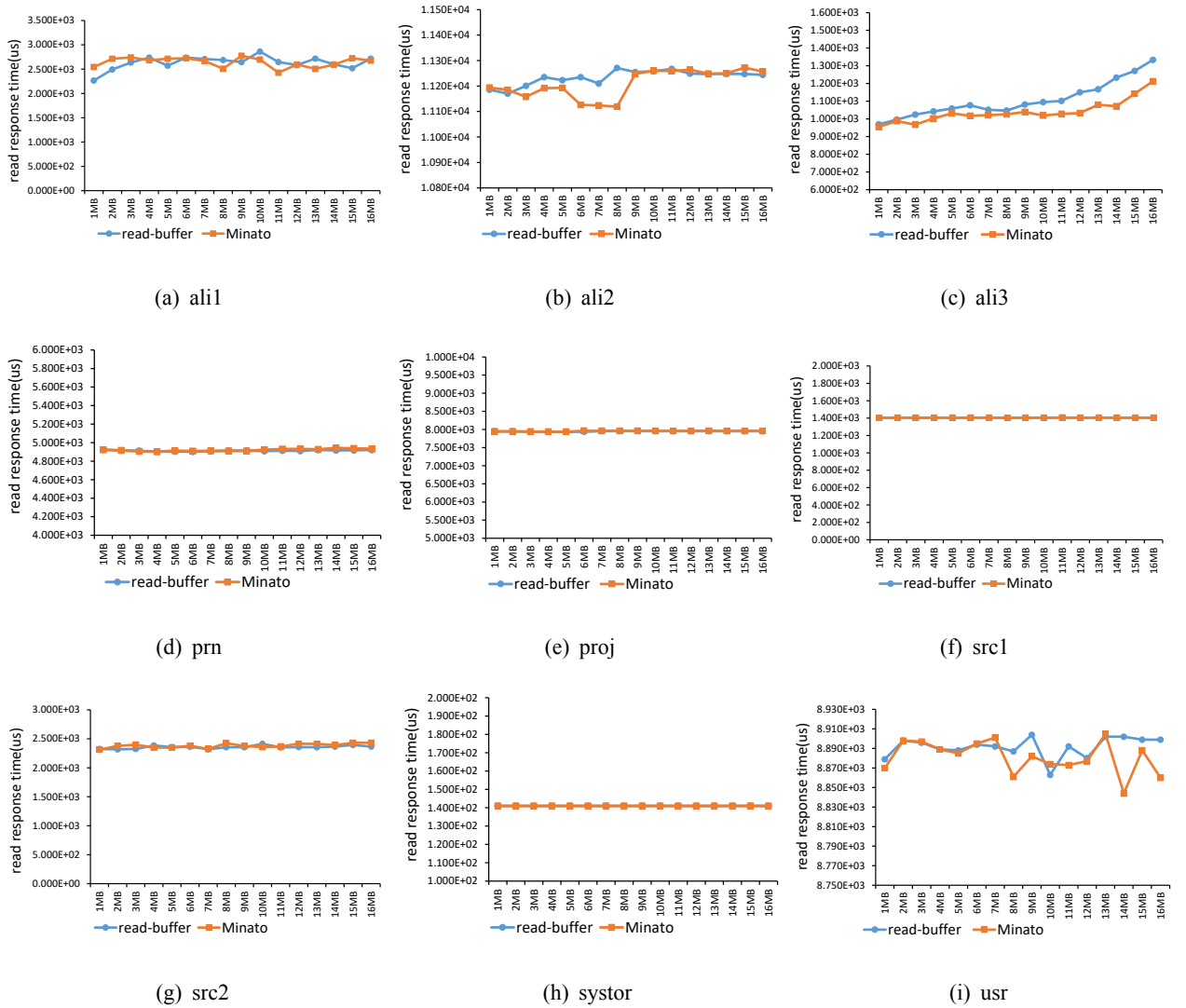


图 4.10 读反应时间情况

Minato 中的读请求过滤机制在热数据较多且读缓存的空间不够时会过滤掉一部分数

据, 优先缓存读干扰影响较大的数据, 从而降低 NAND 闪存整体的读干扰水平。由于 Minato 中存在应对机制 (虚过滤缓存), 过滤数据导致的缓存命中率下降几乎可以忽略。另外, Minato 的主要优化目标是闪存整体的读干扰, 当读干扰下降时, 闪存的读重写次数会降低, 闪存的读反应时间也会降低。图 4.10 展示了九个工作负载的读反应时间对比情况。从图中可以看出, 尽管有些工作负载的读反应时间减少效果较为细微, 但在 ali2、ali3、usr 这几个工作负载中有较为明显的提升效果。

从实验结果可得, 与单纯读缓存情况相比, Minato 可以平均将九个工作负载的读反应时间降低 2.88%, 最高可以降低 13.14% (ali3)。

## 4.5 本章小结

这一章节提出了一种高效、低成本的读干扰感知型数据缓存策略 Minato。通过对 NAND 闪存读干扰影响情况的分析, 提出可以设置一个数据过滤机制, 在读缓存空间十分有限且热读数据较多的情况下, 通过数据过滤机制, 只缓存读干扰影响较大的数据, 也即是有效页面更多的闪存块中的热读数据。介绍了 Minato 中两个主要模块, 分别是读写缓存空间分配机制和读干扰缓解机制。最后在闪存模拟器 MQSim 上对 Minato 进行了实验, 实验结果表明, Minato 可以在不影响写请求命中率的情况下缓解闪存的整体读干扰。与单纯的读缓存策略相比, Minato 的整体读干扰水平、读重写次数、读反应时间最高可减少 3.68%、15.32%、13.14%。

## 5 总结与展望

本文提出在读密集场景下，基于闪存物理特性，从不同结构层级上区别看待读干扰。针对数据不同的访问行为设计了自适应的读干扰感知型放置策略 Cocktail，同时配合垃圾回收机制缓解读干扰，进而提高闪存系统的可靠性。突破了一直以来单纯地通过改变读电压来缓解读干扰的传统方法。进一步的，本文设计了一个读干扰感知型的缓存机制 Minato，将读干扰问题引入闪存系统中的其它数据管理模块。通过准确合理的读干扰预估策略，在闪存系统中建立多层次、多维度的读干扰缓解方案。这将进一步理清闪存读干扰管理策略的内在机理并揭示其规律，完善闪存系统在读干扰缓解方面的方法和理论。

### 5.1 本文工作总结

可靠性是当前闪存系统所面临的最重要的挑战之一。由于大量读密集型场景的出现，读干扰成为了影响闪存系统可靠性不可忽视的主要问题。传统的数据放置策略和数据缓存策略忽略了闪存物理特性的影响。本文提出，在读密集型场景中，如果能综合考虑数据的访问行为和闪存的物理特性对读干扰的影响来在闪存中进行数据放置和数据缓存，则能有效缩短反应时间、减少读干扰，从而减少闪存中额外的擦除操作，延长闪存的使用寿命。在此基础上，本文提出了一种基于访问行为的数据放置策略 Cocktail 和一种基于读干扰感知的数据缓存策略 Minato，用来提高受到读干扰影响的闪存系统的各项性能指标。研究的主要工作如下：

(1) 在基于访问行为的数据放置策略 Cocktail 中，首先分析了传统写入模式下闪存块中的热度分布特点和规律，明确闪存由于数据访问行为而形成热区的形成机理，为利用数据放置来减少闪存块中的热聚集并提高闪存读取效率打下基础；其次通过研究分析出闪存物理特性和闪存可靠性之间的内在联系，探明不同物理位置的读取热度对闪存块电压分布的影响，为设计和优化数据放置策略提供理论支持；之后研究并分析了闪存缓存机制对数据放置策略的影响，揭示数据访问行为、闪存缓存机制、数据放置策略之间的内在规律，重新审视并设计了基于热读场景下的闪存数据放置策略；最后，通过观察分析出了适合以块粒度或者以页粒度进行数据放置的不同应用场景，揭示不同放置方式可能存在的利弊关系，研究并设计了一种适用于多场景的基于闪存物理特性减少闪存块读干扰的自适应数据放置策略。

(2) 在基于读干扰感知的数据缓存策略 Minato 中, 首先研究并分析了不同应用场景下的数据访问行为的内在规律, 构造合适的数学描述模型, 建立数据访问行为的提取、表达、和评估机制, 以便高效地跟踪和预测不同应用场景下的数据访问行为, 从而为设计和优化缓存策略奠定基础; 之后分析了闪存中的缓存机制对闪存读写的影响, 探明在缓存空间中提取数据信息的可能性, 预测未来一段时间窗口内的数据冷热属性; 最后通过衡量基于闪存物理特性和数据访问行为规划不同缓存区域对缓存命中率的影响, 分析根据数据不同写入方式将数据按照不同粒度进行聚集的时效性和合理性, 提出了一种可以按区域特征来吸纳数据的缓存策略。

## 5.2 未来工作展望

闪存的制作工艺和工作原理使其不可避免的存在数据保留丢失、磨损、写放大等问题。为了持续进行存储服务以及避免发生数据丢失, 各大数据中心需要付出大量的维护成本来应对硬盘损耗。因此, 在闪存固态硬盘应用如此广泛的大环境下, 未来研究如何提高闪存寿命和闪存可靠性依然非常具有挑战性。

本文的现有工作中主要提出了一个基于访问行为的数据放置策略和一个基于读干扰感知的数据缓存策略。在数据放置策略中, Cocktail 会根据数据特征动态调整两个阈值, 这有利于将读重写引起的热回写数据分散到更大的块范围内, 从而减少第二次读重写的发生以及平均闪存块的读次数。Cocktail 通过多个工作负载的实验情况为闪存块的预测因子选取了一个定值, 如何根据工作负载访问行为动态调整预测因子有利于提高读重写的预测情况。在数据缓存策略中, Minato 利用了滑动回望窗口来判断当前负载的基本类型, 在之后的工作中可以进行更细致的负载热点周期分析从而提高缓存命中率。Minato 的过滤机制目前只是对有效页面较少的块数据进行过滤, 研究如何结合多队列场景下热读队列和冷读队列的数据存放位置来提供先验知识有利于提高过滤机制的有效性。

## 参考文献

- [1] Masuoka F, Momodomi M, Iwata Y, et al. New ultra high density EPROM and flash EEPROM with NAND structure cell[C]. 1987 International Electron Devices Meeting. 1987: 552-555.
- [2] Haratsch E F. Media management for high density NAND flash memories[J]. Flash Memory Summit, 2016, 3(3): 2-7.
- [3] 陆游游, 杨者, 舒继武. 闪存存储的重构与系统构建技术[J]. 计算机研究与发展, 2019, 56(1): 12.
- [4] Cai Y, Haratsch E F, Mutlu O, et al. Threshold voltage distribution in MLC NAND flash memory: Characterization, analysis, and modeling[C]. 2013 Design, Automation & Test in Europe Conference & Exhibition (DATE). 2013: 1285-1290.
- [5] 陆游游, 舒继武. 闪存存储系统综述[J]. 计算机研究与发展, 2013, 50(1): 11.
- [6] Jaffer S, Mahdavian K, Schroeder B. Improving the Reliability of Next Generation SSDs using WOM-v Codes[C]. 20th USENIX Conference on File and Storage Technologies (FAST 22). 2022: 117-132.
- [7] Park J, Kim M, Chun M, et al. Reducing solid-state drive read latency by optimizing read-retry[C]. Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2021: 702-716.
- [8] 高聪明, 石亮, 刘凯, 等. 闪存固态硬盘系统结构与技术[J]. 计算机研究与发展, 2021, 58(7): 15.
- [9] Wu S, Zhu W, Han Y, et al. GC-Steering: GC-Aware Request Steering and Parallel Reconstruction Optimizations for SSD-Based RAIDs[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2020, 39(12): 4587-4600.
- [10] 吴素贞, 陈晓熹, 毛波. GC-RAIS: 一种基于垃圾回收感知的固态硬盘阵列[J]. 计算机研究与发展, 2013, 50(1): 9.

- [11] Liu C Y, Lee Y, Jung M, et al. Prolonging 3D NAND SSD lifetime via read latency relaxation[C]. Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2021: 730-742.
- [12] Kang M, Park K T, Song Y, et al. Improving read disturb characteristics by self-boosting read scheme for multilevel NAND Flash memories[J]. Japanese Journal of Applied Physics, 2009, 48(4): 76-98.
- [13] Zambelli C, Olivo P, Crippa L, et al. Uniform and concentrated read disturb effects in mid-1X TLC NAND flash memories for enterprise solid state drives[C]. Proceedings of 2017 IEEE International Reliability Physics Symposium (IRPS). 2017: 5.
- [14] Li J, Huang B, Sha Z, et al. Mitigating negative impacts of read disturb in SSDs[J]. ACM Transactions on Design Automation of Electronic Systems (TODAES), 2020, 26(1): 1-24.
- [15] Cai Y, Luo Y, Ghose S, et al. Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery[C]. 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. 2015: 438-449.
- [16] Park K T, Kwon O, Yoon S, et al. A 7MB/s 64Gb 3-bit/cell DDR NAND flash memory in 20nm-node technology[C]. 2011 IEEE International Solid-State Circuits Conference. 2011: 212-213.
- [17] Shim Y, Kim M, Chun M, et al. Exploiting process similarity of 3D flash memory for high performance SSDs[C]. Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. 2019: 211-223.
- [18] Ha K, Jeong J, Kim J. An integrated approach for managing read disturbs in high-density NAND flash memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2015, 35(7): 1079-1091.
- [19] Kang W, Shin D, Yoo S. Reinforcement learning-assisted garbage collection to mitigate long-tail latency in SSD[J]. ACM Transactions on Embedded Computing Systems (TECS), 2017, 16(5): 1-20.
- [20] Zhao M, Li J, Cai Z, et al. Block attribute-aware data reallocation to alleviate read disturb in SSDs[C]. Proceedings of 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE). 2021: 1096-1099.



- [21] Liu C Y, Chang Y M, Chang Y H. Read leveling for flash storage systems[C]. Proceedings of the 8th ACM International Systems and Storage Conference. 2015: 1-10.
- [22] Ha K, Jeong J, Kim J. A read-disturb management technique for high-density NAND flash memory[C]. Proceedings of the 4th Asia-Pacific Workshop on Systems. 2013: 1-6.
- [23] Wu T C, Ma Y P, Chang L P. Flash read disturb management using adaptive cell bit-density with in-place reprogramming[C]. 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE). 2018: 325-330.
- [24] Chang D W, Lin W C, Chen H H. FastRead: Improving read performance for multilevel-cell flash memory[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2016, 24(9): 2998-3002.
- [25] Pletka R, Papandreou N, Stoica R, et al. Improving NAND flash performance with read heat separation[C]. 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS). 2020: 1-8.
- [26] Lv Y, Shi L, Xue C J, et al. Latency variation aware read performance optimization on 3d high density NAND flash memory[C]. Proceedings of the 2020 on Great Lakes Symposium on VLSI. 2020: 411-414.
- [27] Choi W, Jung M, Kandemir M. Invalid data-aware coding to enhance the read performance of high-density flash memories[C]. 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2018: 482-493.
- [28] Long L, Huang J, Gao C, et al. ADAR: Application-specific Data Allocation and Reprogramming Optimization for 3D TLC Flash Memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023, 1(1): 1-14.
- [29] Shim H, Seo B K, Kim J S, et al. An adaptive partitioning scheme for DRAM-based cache in solid state drives[C]. 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST). 2010: 1-12.
- [30] Hu Y, Jiang H, Feng D, et al. PASS: A proactive and adaptive SSD buffer scheme for data-intensive workloads[C]. 2015 IEEE International Conference on Networking, Architecture and Storage (NAS). 2015: 54-63.

- [31] Yeh P C, Wu C H, Lin Y H, et al. A Write-Related and Read-Related DRAM Allocation Strategy Inside Solid-State Drives (SSDs)[J]. *ACM Transactions on Embedded Computing Systems*, 2022, 22(1): 1-32.
- [32] Megiddo N, Modha D S. ARC: A Self-Tuning, Low Overhead Replacement Cache.[C]. *FAST*: vol. 3: 2003. 2003: 115-130.
- [33] Jiang S, Zhang X. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance[J]. *ACM SIGMETRICS Performance Evaluation Review*, 2002, 30(1): 31-42.
- [34] Kim H, Ahn S. BPLRU: A Buffer Management Scheme for Improving Random Writes in Flash Storage.[C]. *FAST*: vol. 8. 2008: 1-14.
- [35] Wu S, Mao B, Lin Y, et al. Improving performance for flash-based storage systems through GC-aware cache management[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2017, 28(10): 2852-2865.
- [36] Wu G, He X, Eckart B. An adaptive write buffer management scheme for flash-based ssds [J]. *ACM Transactions on Storage (TOS)*, 2012, 8(1): 1-24.
- [37] Mielke N, Marquart T, Wu N, et al. Bit error rate in NAND flash memories[C]. *2008 IEEE International Reliability Physics Symposium*. 2008: 9-19.
- [38] Shibata N, Kanda K, Shimizu T, et al. A 1.33-Tb 4-bit/cell 3-D flash memory on a 96-word-line-layer technology[J]. *IEEE Journal of Solid-State Circuits*, 2019, 55(1): 178-188.
- [39] Elyasi N, Arjomand M, Sivasubramaniam A, et al. Exploiting intra-request slack to improve SSD performance[C]. *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 2017: 375-388.
- [40] Tavakkol A, Mehrvarzy P, Arjomand M, et al. Performance evaluation of dynamic page allocation strategies in SSDs[J]. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)*, 2016, 1(2): 1-33.
- [41] Gupta A, Kim Y, Urgaonkar B. DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings[J]. *ACM Sigplan Notices*, 2009, 44(3): 229-240.

- [42] Chang L P, Kuo T W, Lo S W. Real-time garbage collection for flash-memory storage systems of real-time embedded systems[J]. *ACM Transactions on Embedded Computing Systems (TECS)*, 2004, 3(4): 837-863.
- [43] Cai Y, Luo Y, Haratsch E F, et al. Data retention in MLC NAND flash memory: Characterization, optimization, and recovery[C]. *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. 2015: 551-563.
- [44] Luo Y, Ghose S, Cai Y, et al. Enabling accurate and practical online flash channel modeling for modern MLC NAND flash memory[J]. *IEEE Journal on Selected Areas in Communications*, 2016, 34(9): 2294-2311.
- [45] Cai Y, Yalcin G, Mutlu O, et al. Flash correct-and-refresh: Retention-aware error management for increased flash memory lifetime[C]. *2012 IEEE 30th International Conference on Computer Design (ICCD)*. 2012: 94-101.
- [46] Cai Y, Yalcin G, Mutlu O, et al. Neighbor-cell assisted error correction for MLC NAND flash memories[J]. *ACM SIGMETRICS Performance Evaluation Review*, 2014, 42(1): 491-504.
- [47] Cai Y, Haratsch E F, Mutlu O, et al. Error patterns in MLC NAND flash memory: Measurement, characterization, and analysis[C]. *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2012: 521-526.
- [48] Lee Y, Yoo H, Yoo I, et al. 6.4 Gb/s multi-threaded BCH encoder and decoder for multi-channel SSD controllers[C]. *2012 IEEE International Solid-State Circuits Conference*. 2012: 426-428.
- [49] Bose R C, Ray-Chaudhuri D K. On a class of error correcting binary group codes[J]. *Information and Control*, 1960, 3(1): 68-79.
- [50] MacKay D J, Neal R M. Near Shannon limit performance of low density parity check codes [J]. *Electronics Letters*, 1997, 33(6): 457-458.
- [51] Zhao K, Zhao W, Sun H, et al. LDPC-in-SSD: Making advanced error correction codes work effectively in solid state drives[C]. Presented as part of the 11th USENIX Conference on File and Storage Technologies (FAST 13). 2013: 243-256.

- [52] Peterson W W, Brown D T. Cyclic codes for error detection[J]. Proceedings of the IRE, 1961, 49(1): 228-235.
- [53] Dirik C, Jacob B. The performance of PC solid-state disks (SSDs) as a function of bandwidth, concurrency, device architecture, and system organization[J]. ACM SIGARCH Computer Architecture News, 2009, 37(3): 279-289.
- [54] Patterson D A, Gibson G, Katz R H. A case for redundant arrays of inexpensive disks (RAID) [C]. Proceedings of the 1988 ACM SIGMOD international conference on Management of data. 1988: 109-116.
- [55] Prabhu P, Akel A, Grupp L M, et al. Extracting device fingerprints from flash memory by exploiting physical variations[C]. Trust and Trustworthy Computing: 4th International Conference. 2011: 188-201.
- [56] Parnell T, Papandreou N, Mittelholzer T, et al. Modelling of the threshold voltage distributions of sub-20nm NAND flash memory[C]. 2014 IEEE Global Communications Conference. 2014: 2351-2356.
- [57] Compagnoni C M, Ghidotti M, Lacaita A L, et al. Random telegraph noise effect on the programmed threshold-voltage distribution of flash memories[J]. IEEE Electron Device Letters, 2009, 30(9): 984-986.
- [58] Ghatti A, Compagnoni C M, Spinelli A S, et al. Comprehensive analysis of random telegraph noise instability and its scaling in deca-nanometer flash memories[J]. IEEE Transactions on Electron Devices, 2009, 56(8): 1746-1752.
- [59] Joe S M, Yi J H, Park S K, et al. Threshold voltage fluctuation by random telegraph noise in floating gate NAND flash memory string[J]. IEEE Transactions on Electron Devices, 2010, 58(1): 67-73.
- [60] Fukami A, Ghose S, Luo Y, et al. Improving the reliability of chip-off forensic analysis of NAND flash memory devices[J]. Digital Investigation, 2017, 20: 1-11.
- [61] Liu W, Wu F, Zhang M, et al. DEPS: Exploiting a Dynamic Error Prechecking Scheme to Improve the Read Performance of SSD[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2021, 40(1): 66-77.

- [62] Lv Y, Shi L, Luo L, et al. Tail Latency Optimization for LDPC-Based High-Density and Low-Cost Flash Memory Devices[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022, 41(3): 544-557.
- [63] Yang F, Chen Y, Mao H, et al. ShieldNVM: An efficient and fast recoverable system for secure non-volatile memory[J]. ACM Transactions on Storage (TOS), 2020, 16(2): 1-31.
- [64] Van Houdt, Benny. Performance of garbage collection algorithms for flash-based solid state drives with hot/cold data[J]. Performance Evaluation, 2013, 70(10): 692-703.
- [65] Chang L P, Kuo T W. An adaptive striping architecture for flash memory storage systems of embedded systems[C]. Proceedings of the 8th IEEE Real-Time and Embedded Technology and Applications Symposium. 2002: 187-196.
- [66] Chiang M L, Lee P C, Chang R C. Using data clustering to improve cleaning performance for flash memory[J]. Software: Practice and Experience, 1999, 29(3): 267-290.
- [67] Van Houdt B. On the necessity of hot and cold data identification to reduce the write amplification in flash-based SSDs[J]. Performance Evaluation, 2014, 82: 1-14.
- [68] Hu X Y, Eleftheriou E, Haas R, et al. Write amplification analysis in flash-based solid state drives[C]. Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference. 2009: 1-9.
- [69] Desnoyers P. Analytic modeling of SSD write performance[C]. Proceedings of the 5th Annual International Systems and Storage Conference. 2012: 1-10.
- [70] Li J, Wang Q, Lee P P, et al. An In-Depth Analysis of Cloud Block Storage Workloads in Large-Scale Production[C]. Proceedings of 2020 IEEE International Symposium on Workload Characterization (IISWC). 2020: 37-47.
- [71] Lee C, Kumano T, Matsuki T, et al. Understanding storage traffic characteristics on enterprise virtual desktop infrastructure[C]. Proceedings of the 10th ACM International Systems and Storage Conference. 2017: 1-11.
- [72] Zhou D, Pan W, Wang W, et al. I/O characteristics of smartphone applications and their implications for eMMC design[C]. Proceedings of 2015 IEEE International Symposium on Workload Characterization. 2015: 12-21.

- [73] Kavalanekar S, Worthington B, Zhang Q, et al. Characterization of storage workload traces from production windows servers[C]. Proceedings of 2008 IEEE International Symposium on Workload Characterization. 2008: 119-128.
- [74] Narayanan D, Donnelly A, Rowstron A. Write off-loading: Practical power management for enterprise storage[J]. ACM Transactions on Storage (TOS), 2008, 4(3): 1-23.
- [75] Kim C, Kim D H, Jeong W, et al. A 512-gb 3-b/cell 64-stacked wl 3-d-nand flash memory [J]. IEEE Journal of Solid-State Circuits, 2017, 53(1): 124-133.
- [76] Lee S, Kim C, Kim M, et al. A 1Tb 4b/cell 64-stacked-WL 3D NAND flash memory with 12MB/s program throughput[C]. Proceedings of 2018 IEEE International Solid-State Circuits Conference. 2018: 340-342.
- [77] Tavakkol A, Gómez-Luna J, Sadrosadati M, et al. Mqsim: A framework for enabling realistic studies of modern multi-queue SSD devices[C]. 16th USENIX Conference on File and Storage Technologies (FAST 18). 2018: 49-66.
- [78] Van Houdt B. A mean field model for a class of garbage collection algorithms in flash-based solid state drives[J]. ACM SIGMETRICS Performance Evaluation Review, 2013, 41(1): 191-202.
- [79] Zhang G, Deng Y, Zhou Y, et al. Cocktail: Mixing data with different characteristics to reduce Read Reclaims for NAND Flash Memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023, 1(1): 1-14.
- [80] Cai Y, Ghose S, Haratsch E F, et al. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives[J]. Proceedings of the IEEE, 2017, 105(9): 1666-1704.
- [81] Xiong Q, Wu F, Lu Z, et al. Characterizing 3D floating gate NAND flash: Observations, analyses, and implications[J]. ACM Transactions on Storage (TOS), 2018, 14(2): 1-31.
- [82] Liu W, Wu F, Zhang M, et al. Characterizing the reliability and threshold voltage shifting of 3D charge trap NAND flash[C]. 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE). 2019: 312-315.

## 在学期间发表的学术论文及科研成果清单

### 学术论文

- [1] Zhang G, Deng Y, Zhou Y, et al. Cocktail: Mixing data with different characteristics to reduce Read Reclaims for NAND Flash Memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems. (CCF 推荐 A 类期刊, 已接收)
- [2] Pang S, Deng Y, Zhang G, et al. PSA-Cache: A Page-State-Aware Cache Scheme for boosting 3D NAND Flash Performance[J]. ACM Transactions on Storage. (CCF 推荐 A 类期刊, 已接收)

### 发明专利

- [1] 邓玉辉, **张根雄**。利用数据不同特征延长 SSD 寿命的方法、装置、设备及介质, 公开阶段, 申请号: CN202210954690.9, 公布号: CN115240743A。

## 致 谢

此时我正坐在 612 实验室的工位上，耳边是师兄师弟们讨论的声音，开始敲下这段致谢词。预答辩和盲审已经过去了一段时间，我一直拖着没写致谢，仿佛致谢写完，这三年的学习生活也会落下帷幕。所以我给自己定了个小目标，写完第二篇小论文之后再来写致谢。庆幸拖延症患者今天终于把小论文写完过了一遍交给老师（粗制滥造后面还需要很多修改），总算可以安心毕业了。

回顾硕士这三年，课程、比赛、选题、论文、实习、秋招、毕业，一步步走来都是满满的收获。还记得打华为比赛的时候，和吕飘誉炫在实验室不断讨论最优化方案，常常同一个思路要写好几种代码实现更低的开销和更优的效果，一夜夜的失眠，脑子里不断复盘旧方案 and 对比新方案；还记得准备第一篇论文时不知如何下手，第一版写得一塌糊涂，后面在一次次修改中明白了一篇论文中必须要有非常明确的动机，由这个动机去牵引写作；还记得在杭州字节实习的时候，论文大修忽然而至，在杭州城燥热的夏天里，白天到公司上班，晚上回出租屋继续改代码、修论文，感觉时间和精力被完全榨干。过程虽然辛苦，好在非常幸运，各个阶段都有比较好的结果，逐步接近自己读研的目标：顺利毕业！

毕业之后就是一个全新的战场，这么多年的学校生活也要画上一个完满的句号了。要特别感谢我的父母这些年在各个方面给予我的信任、支持和帮助，让我可以安心完成学业；感谢陪伴我一路走来的同学们和在实验室一起学习、一起玩闹的师兄师姐师弟师妹，我最宝贵的财富就是你们这样一群同路人；感谢我的导师邓玉辉教授在学业上对我的帮助，每次在我对自己的成果缺乏信心时总能给予我必要的支持和鼓励，让我能够继续去坚持完善实验和论文。最后要特别感谢脚踏实地、一步一个脚印走过来的自己，我始终相信，无需着眼于飘渺的结果，认准一件事后只管不遗余力去解决眼前的困难，最后结局总不会太差。

我将永远怀念暨南园的一切。真心祝愿所有老师和同学身体健康，科研有成！

张根雄

2023 年 5 月 23 日于暨南园